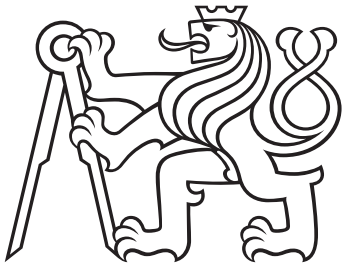
České vysoké učení technické v Praze
Fakulta elektrotechnická
Katedra měření
Laboratoř videometrie



Možnosti použití balíku STM32duino

Vypracoval: Stanislav Novák, novaks11@cvut.cz
Vedoucí práce: doc. Ing. Jan Fischer, CSc
Verze: 5. května 2026

0.1 Úvod

STM32duino je softwarový balík rozšiřující softwarové nástroje Arduina o možnost programování mikrokontrolérů STM32. Arduino je otevřená platforma jednodeskových počítačů založených původně na mikrokontrolérech ATmega. STM32duino začínalo jako komunitní projekt, v současnosti je velkou měrou udržovaný firmou ST.

Balík jako takový se nachází na githubu (github.com/STM32duino). Odtud je možné stáhnout zdrojové kódy a nalézt zde dokumentaci. Jako další zdroj informací existuje diskuzní fórum www.STM32duino.com, případně je možné nalézt informace i jinde (např. na community.st.com). Na webech nadšenců lze také nalézt základní návody, obvykle však nejdou výrazněji do hloubky (např. od Petra Šrámka na chiptron.cz).

Cílem této práce je lépe balík zdokumentovat z uživatelského pohledu, protože dostupná dokumentace není dostatečná a obsahuje v mnoha případech neaktuální informace.

1 Základy STM32duina, instalace

1.1 Arduino

Arduino vzniklo jako školní projekt pro výuku programování. Mikrokontroléry byly využity jako příklad praktického a pochopitelného užití programu ("hmatatelnost"pomocí LED, tlačítek a dalších prvků). Arduino bylo zprvu založeno na mikrokontrolérech ATmega osazených na deskách pouze s minimálním zapojením. Zapojení takovýchto desek se ukázalo jako velmi užitečné v praxi při vývoji elektroniky; výrobci komponent tento koncept často převzali – např. desky Nucleo od ST. Takovéto zapojení rovněž oceňují kutilové při svých projektech.

Arduino používá pro programování jazyk C (přesněji C++) rozšířené o knihovny pro jednoduché ovládání hardware (např. pro nastavení obdélníkový signál jisté frekvence na pinu funkcí s pouze dvěma argumenty). Tyto knihovny jsou často označovány jako jazyk Wiring. Ke psaní programů, kompilaci a nahrávání slouží nástroj Arduino IDE, při jeho vývoji byla upřednostňována jednoduchost ovládání.

Pro STM32duino je použita pouze softwarová část, konkrétně IDE a jazyk Wiring. Samotné IDE je možné stáhnout z webu www.arduino.cc/en/software; v tuto chvíli jsou dostupné dvě verze (1.y.z a 2.y.z). Pro účely STM32duina je výhodnější použití verze 2 ve které je možnost ladění pomocí SWD. Zároveň, v nových verzích STM32duina byla již ukončena podpora IDE verze 1. Arduino IDE je multiplatformní, existují nativní verze pro Linux i macOS. Samotná instalace je přímočará (pro úplnost uvádíme odkaz na postup na arduino.cc). Samotné Arduino je velmi populární a v případě problému lze obvykle dohledat obrovské množství rad a návodů volně na internetu.

Dobře zpracovanou dokumentaci jazyka Wiring lze nalézt na webu arduino.cc. Podle této specifikace se mají chovat dokumentované funkce i v STM32duinu (rozumějte ve významu "emulace"chování ATmegy).

1.2 Instalace STM32duina, popis nastavení

1.2.1 Instalace Arduina

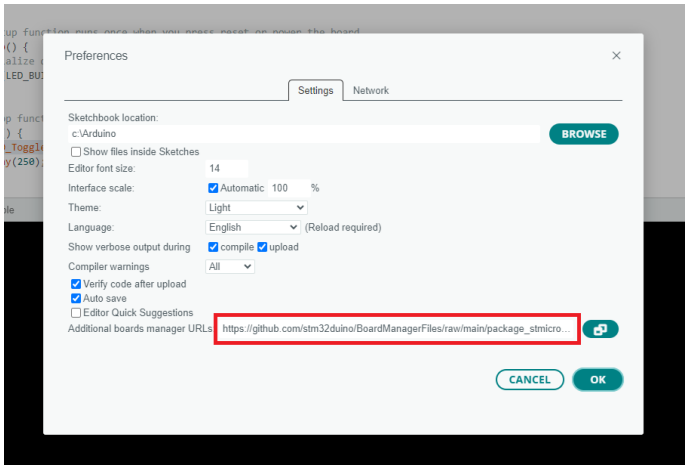
Před instalací STM32duina je nutné provést instalaci Arduino IDE. Samotné IDE lze stáhnout z webu www.arduino.cc/en/software. V tuto chvíli jsou dostupné dvě verze (1.y.z a 2.y.z). Pro účely STM32duina je výhodnější použití verze 2, ve které je možnost ladění pomocí SWD. Zároveň od STM32duina verze 2.8.0 je ukončena podpora IDE verze 1.

Arduino IDE je multiplatformní, existují nativní verze pro Linux i macOS. Samotná instalace je přímočará (pro úplnost uvádíme odkaz na postup na arduino.cc). Samotné Arduino je velmi populární a v případě potíží lze obvykle dohledat značné množství relevantních rad a návodů volně na internetu.

1.2.2 Instalace STM32duina

Po instalaci Arduino IDE je třeba nainstalovat samotné STM32duino. Samotný postup lze nalézt na githubu [1] (v angličtině). Pro účel této práce je v této kapitole postup taktéž uveden (přeložen do češtiny, upraveno).

1. Nejdříve je třeba v nabídce **File** → **Preferences** nastavit adresu `https://github.com/stm32duino/BoardManagerFiles/raw/main/package_stmicroelectronics_index.json` do kolonky **Additional Boards Managers URLs:** (zvýrazněno na obrázku 1).



Obrázek 1: Okno **Preferences** aplikace Arduino IDE

2. Dále je třeba v nabídce **Tools** → **Board:** → **Boards Manager...** (obrázek 2, vlevo) nainstalovat balík **STM32 MCU based boards:** (obrázek 2, vpravo; vhodné je využít filtr) tlačítkem **INSTALL**.

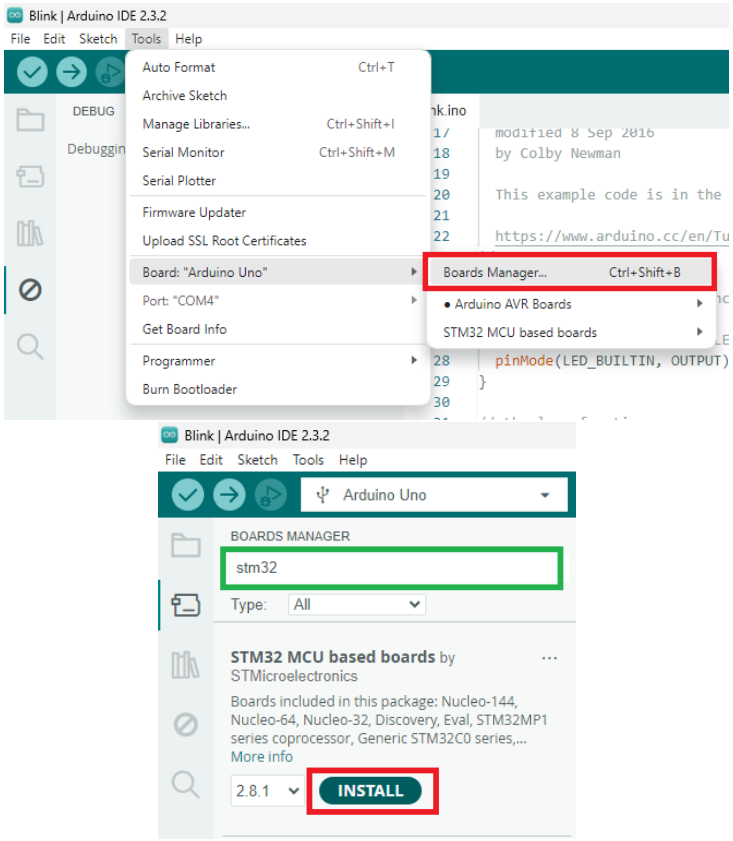
3. Po instalaci je třeba nastavit použitou vývojovou desku, a to ve dvou krocích. V prvním kroku se nastavuje skupina desek, např. Nucleo, nebo generická deska skrze **Tools** → **Board:** → **STM32 MCU based boards** (obrázek 3, vlevo; **Generic STM32G0 series** jako příklad). V druhém kroku se ze skupiny volí konkrétní deska skrze **Tools** → **Board part number:** (obrázek 3, vpravo; **STM32G030J6** jako příklad).

4. Nakonec je vhodné nastavit metodu nahrávání skrze **Tools** → **Upload Method** (závisí na konkrétní desce, která možnost je dostupná/ideální). Některé metody nahrávání (označné **STM32CubeProgrammer**) potřebují doinstalovat program STM32CubeProg (dostupný z st.com, postup instalace zde není uveden, je přímočarý, zároveň ST poskytuje dostatečnou dokumentaci).

1.2.3 Možnosti nastavení STM32duina v Arduino IDE

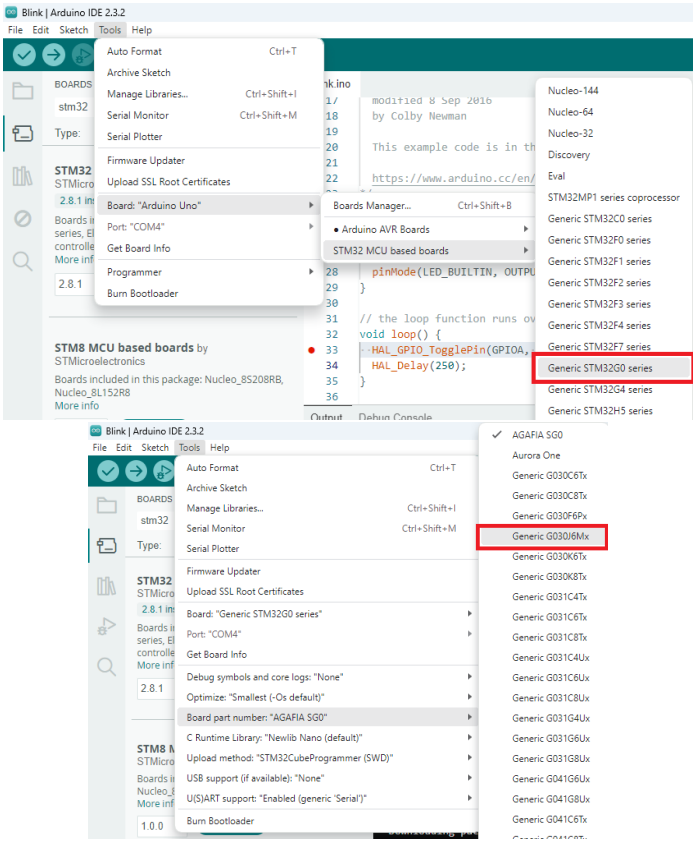
Po instalaci je vhodné představit nastavení, která STM32duino do Arduino IDE přináší. Všechna nastavení jsou součástí nabídky **Tools**. Na obrázku 4 je zobrazena nabídka s jednotlivými nastaveními. Pro úplnost je vhodné jednotlivá nastavení popsat v následujícím seznamu:

- **Board:** Zde se vybírá skupina cílového zařízení; takovou skupinou může být deska *Nucleo*, *generická deska* (jakákoli uživatelem navržená deska obsahující mikrokontrolér dané řady), případně jiná skupina desek. Toto nastavení již bylo ukázáno v předchozí kapitole.



Obrázek 2: Instalace balíku

- **Port:** Zde je vybírán port COM port použitý pro komunikaci (používá se pro uživatelské výpisy, případně pro nahrávání programu). Tento je třeba typicky správně nastavit v případě použití UARTu, USB třídy CDC (Virtual COM port), nebo při nahrávání firmware bootloaderem periferií USART.
- **Get Board Info** Vypíše systémové informace o vybraném COM portu; v případně virtuálního COM portu ST-Linku se zobrazí jeho sériové číslo (může být v některých případech užitečné).
- **Debug symbols and core logs:** Toto nastavení je důležité při ladění ST-Linkem; určuje, zda symboly jsou součástí výstupu kompilátoru. Při ladění je třeba nastavit na **Symbols Enabled (-g)**
- **Optimize:** Nastavení optimalizace při kompilaci (standardní nastavení optimalizace kompilátoru GCC, vhodné nastavit na **Smallest -Os**).
- **Board part number:** Zde se nastavuje konkrétní cílové zařízení pro které se má sestavit program. Možná nastavení se řídí nastavením **Board:**. Toto nastavení již bylo ukázáno v předchozí kapitole.
- **C Runtime Library:** Pokročilé nastavení knihoven C převážně relevantní pro výpisy; typicky není třeba měnit.
- **Upload method:** Určuje jakým způsobem se nahrává program. K tomuto existuje dokumentace dostupná na githubu. Metoda **Mass Storage** používá možnosti ST-Linku V2.1 pro



Obrázek 3: Nastavení desky

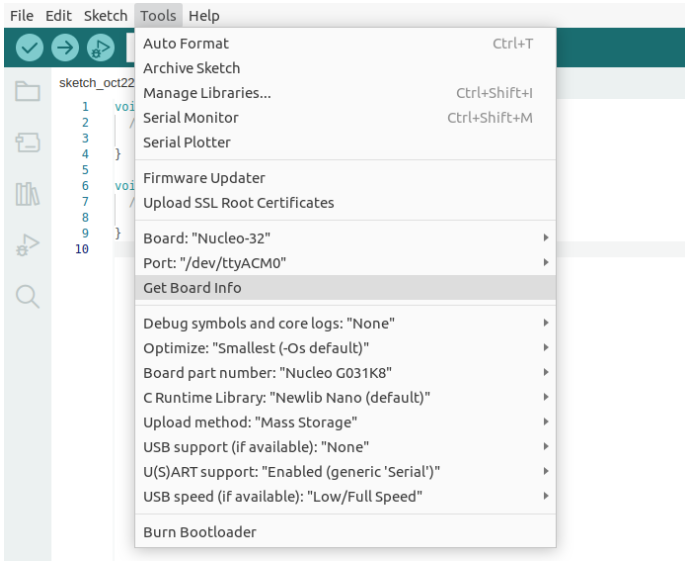
nahrávání programu bez externích nástrojů. Ostatní metody používají externí nástroj (označené **STM32CubeProgrammer**), jednotlivá nastavení ukazují, které periferie jsou k nahrávání použity. Novou možností (od v2.9.0) je **OpenOCD STLink (SWD)**, které umožňuje nahrávání pomocí SWD bez nástroje STM32CubeProgrammer (pomocí OpenOCD využitého pro debug).

- **USB support (if available):** Pokud se používá periferie USB, je třeba zde nastavit, jaká třída (CDC, nebo HID) má být použita pro vložení odpovídající knihovny.
- **U(S)ART support:** Nastavení určující, zda se budou vkládat knihovny pro USART. Pokud USART nebude v programu použit (např. pokud je namísto toho použito USB CDC), je vhodné vypnout (možností **Disabled (no Serial support)**) pro zmenšení velikosti programu.
- **USB speed (if available):** Pokročilé nastavení pro STM32 s rychlejšími USB periferiemi než Full-speed.

1.2.4 Prostředí Arduino IDE, příklady programů

Samotné prostředí Arduin IDE je vcelku dobře dokumentované v internetové dokumentaci na (`https://docs.arduino.cc/software/ide/#ide-v2`), základní popis a první kroky jsou popsány na `https://docs.arduino.cc/learn/starting-guide/the-arduino-software-ide/`.

Arduino IDE má vcelku elegantně vyřešen přístup k příkladům programů, konkrétně skrze nabídku **File** → **Examples**. V nabídce jsou příklady pro lepší orientaci seskupeny, pro STM32duino



Obrázek 4: Nastavení STM32duina

jsou důležité jeho příklady ze skupiny příslušné skupině desek (např. **Examples for Generic STM32G0 series**).

1.2.5 Nahrávání programů

Před nahráváním je třeba správně nastavit metodu nahrávání (skrze **Tools** → **Upload Method**, viz předcházející kapitoly). Samotné nahrávání lze provést pomocí skrze nabídku **Sketch** → **Upload**, nebo intuitivněji tlačítkem šipky (→) na liště. Jako vhodné se jeví uvést možné chyby při kompilaci s krátkým vysvětlením:

- **Compilation error: ...** (typicky) Chyba v programu – program nedává z pohledu jazyka C smysl (typicky překlep, např. chybějící středník).
- **Failed uploading: no upload port provided**, nebo **Failed uploading: Cannot open port...** Nesprávně nastavený port. Port lze nastavit v **Tools** → **Port** Specifické problémy mohou nastat při použití UART-USB převodníků CH340, tyto potřebují doinstalovat ovladače (`https://www.wch-ic.com/downloads/category/30.html`). Další specifický problém nastává na Linuxu, kdy uživatel nemá přístup k portům (typicky je třeba uživatele přidat do skupiny `dialout`).
- **„...“ not found** (např. **NOD_G031K8 not found**). Pravděpodobně je zvolena metoda **Mass storage** a není připojeno tomuto odpovídající deska.
- **STM32CubeProg not found...**, STM32CubeProg není nainstalován, nebo prostředí k němu nezná cestu. Řešení je popsáno na githubu.

Další uvedené chyby jsou spojeny se STM32CubeProg (jedná se přímo o výpis STM32CubeProg):

- **Error: Activating device: KO...**, tato chyba indikuje, že se podařilo otevřít COM port, ale mikrokontrolér STM32 neodpovídá. V tuto chvíli je dobré zkontrolovat, který COM port je nastaven, dále zapojení UARTU (Rx-Tx), případně ověřit, zda je zařízení v bootloaderu (ověřit stav pinu/bitu BOOT0; kompletní specifikace pro bootloader je aplikační nóta AN2606 [2]).

- **Error: No debug probe detected**, je nastavena nahrávací metoda SWD (ST-Link) a žádný ST-Link není připojen.
- **Error: Target device not found**, je nastavena nahrávací metoda USB-DFU a žádné odpovídající zařízení nebylo nalezeno.

Pro konkrétní ověření nastavení metody nahrávání je dobré použít jednoduchý příklad, např. blikání LED z následující kapitoly.

1.3 Základní funkce a příklady

1.3.1 Digitální vstupy a výstupy

Jako tradiční první příklad se uvádí blikání LED. Blikání LED typicky ověřuje, že je možné do zařízení nahrát vlastní program. V Arduino IDE lze otevřít příklad **Basics** → **Blink** (příklady se nacházejí ve **File** → **Examples**), případně přepsat kód z této kapitoly (použitý pin odpovídá typické desce Nucleo-64, např. NUCLEO-G071RB). Pro použití na generické (uživatelské) desce je třeba změnit v kódu jméno pinu (nebo makro `LED_BUILTIN`) na jméno odpovídající připojené LED (v sérii s rezistorem připojeným k zemi).

```
1  void setup() {
2    pinMode(PA5, OUTPUT);
3  }
4
5  void loop() {
6    digitalWrite(PA5, HIGH);
7    delay(100);
8    digitalWrite(PA5, LOW);
9    delay(100);
10 }
```

Listing 1: Blikaní LED

Samotné Arduino skrývá standardní funkci `main()`, za funkce `setup()` (jednou se spustí na začátku) a `loop()` (po skončení funkce `setup()` se spouští v cyklu donekonečna). Důležitým rozdílem oproti Arduinu je nutnost vždy iniciovat pin (funkcí `pinMode()`). Pokud bychom chtěli číst digitální hodnotu pinu (např. tlačítka), je třeba iniciovat pin jako digitální vstup (`INPUT`) a poté lze číst hodnotu funkcí `digitalRead()`. Jako příklad uvádíme „blikání při stisku tlačítka“ (pro NUCLEO-G071RB, ale snadno přenositelné).

```
1  void setup() {
2    pinMode(PC13, INPUT);
3    pinMode(LED_BUILTIN, OUTPUT);
4  }
5
6  void loop() {
7    if(digitalRead(PC13)==LOW){
8      digitalWrite(PA5, HIGH);
9      delay(100);
10     digitalWrite(PA5, LOW);
11     delay(100);
12   }
13 }
```

Listing 2: Blikaní LED s tlačítkem

Tento příklad je validní, pokud je k tlačítku připojen externí pull-up rezistor. Pokud tlačítko při stisknutí pouze „zkratovává“ pin k zemi, je třeba zapnout pull-up rezistor na čipu (je třeba upravit druhý řádek na `pinMode(PC13, INPUT_PULLUP);`).

1.3.2 UART (sériová linka)

UART je rozšířeným rozhraním pro komunikaci mezi počítači, pro Arduino jako takové má typicky UART dvě využití :

1. Nahrávání programu
2. Uživatelské výpisy/ovládání programu

Na STM32 je typicky realizován periferií USART podporující jak asynchronní, tak synchronní přenos, pro tuto chvíli se však omezme na asynchronní variantu. Pro ověření funkce výpisů je možné použít příklad **Communication → ASCIITable** obsažený v samotném Arduinu. Po nahrání programu a otevření sériového terminálu (v **Tools → Serial Monitor**) by se měl zobrazit výpis části ASCII tabulky. Při nastavování UARTU je nutné dodržet nastavení baud rate a dále mít nastavený správný port počítače (v **Tools → Port**).

Při posílání dat do desky je typicky použita funkce `Serial.available()` (vrací kolik znaků je nabufferovaných v paměti) a `Serial.read()` (vrací znak vyčtený z bufferu). Samotné použití lze pochopit z následujícího příkladu echa (přijatá data se posílají zpět):

```
1 void setup() {
2   Serial.begin(115200);
3 }
4
5 void loop() {
6   if(Serial.available()) Serial.write(Serial.read());
7 }
```

Listing 3: UART echo

Užitečné může být přečtení dokumentace k jednotlivým funkcím objektu `Serial` na `https://www.arduino.cc/reference/en/language/functions/communication/serial/`.

Při použití jiných desek, než Nucleo, je také třeba dohledat, na kterých pinech se nachází výchozí UART (viz kapitola „Definice desek“ 1.4.2), případně příklad upravit (pro použití pinu některé instance USARTx). Při úpravě je typicky vhodné nahradit výchozí objekt `Serial` vlastním objektem `mySerial` (je možné pojmenovat téměř libovolně), kterému v inicializaci nastavíme, které piny má používat.

Pozor, LPUART problém: LPUART je varianta periferie UART, která může použít jiné zdroje hodin než sběrnici. Toho lze s výhodou využít – při použití LSE (krystalu přesného času 32768 Hz) lze dosáhnout 9600 baud, což může být výhodné při režimech snížené spotřeby. STM32duino však typicky předpokládá, že LSE je dostupné – proto při 9600 baud u této periferie nastaví LSE jako zdroj hodin (**pokud LSE není pak nepracuje**). Zároveň problém se projevuje díky faktu, že definice jsou generované z dat STM32CubeMX a „LPUART je v abecedě dříve než UARTx“.

Řešení LPUART problém: Výčet možných řešení je následující:

1. Zvyšte baud rate (na např. 115200) aby se nepoužilo LSE jako zdroj hodin.
2. Podívejte se do definic a nastavte piny na ne-LP UART. Konkrétně otevřete soubor **PeripheralPins.c** (popis definicí kapitola „Definice desek“ 1.4.2), kde může být např. následující definice:

```
WEAK const PinMap PinMap_UART_TX[] = {
    {PA_2,      LPUART1, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_UP, GPIO_AF6_LPUART1)},
    {PA_2_ALT1, USART2,  STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_UP, GPIO_AF1_USART2)},
...
```

Pro nastavení pinu PA2 pro LPUART1 použijeme `PA2`, pro nastavení PA2 pro USART2 použijeme `PA2_ALT1` (obdobně pro Rx pin).

Konkrétní upravený příklad **ASCIITable** může vypadat následovně (porovnejte s výchozím příkladem z knihovny [3]):

```
1  HardwareSerial mySerial(PA10, PA9);//Rx,Tx
2
3  void setup() {
4    //Initialize serial and wait for port to open:
5    mySerial.begin(115200);
6    while (!mySerial) {
7      ;  // wait for the serial port to connect. Needed for native
       USB port only
8    }
9
10   // prints title with ending line break
11   mySerial.println("ASCII Table ~ Character Map");
12 }
13
14 // first visible ASCIIcharacter '!' is number 33:
15 int thisByte = 33;
16 // you can also write ASCII characters in single quotes.
17 // for example, '!' is the same as 33, so you could also use this:
18 // int thisByte = '!';
19
20 void loop() {
21   // prints value unaltered, i.e. the raw binary version of the
       byte.
22   // The mySerial Monitor interprets all bytes as ASCII, so 33, the
       first number,
23   // will show up as '!'
24   mySerial.write(thisByte);
25   // if printed last visible character '~' or 126, stop:
26   if (thisByte == 126) {  // you could also use if (thisByte ==
       '~') {
27     mySerial.write(10);
28     thisByte = 32;
29   }
30   while(mySerial.available()) mySerial.write(mySerial.read());//
       echo
31   // go on to the next character
32   thisByte++;
33   delay(100);
34 }
```

Listing 4: Upravený příklad **ASCII table**

Úprava, která byla provedena v příkladu **ASCIITable** (aby bylo možné použít pro UART jiné piny, než výchozí) využívala knihovnu **HardwareSerial** STM32duina. Dokumentace k této knihovně je dostupná na `https://github.com/STM32duino/Arduino_Core_STM32/wiki/API#hardwareserial`).

1.3.3 Analogové čtení

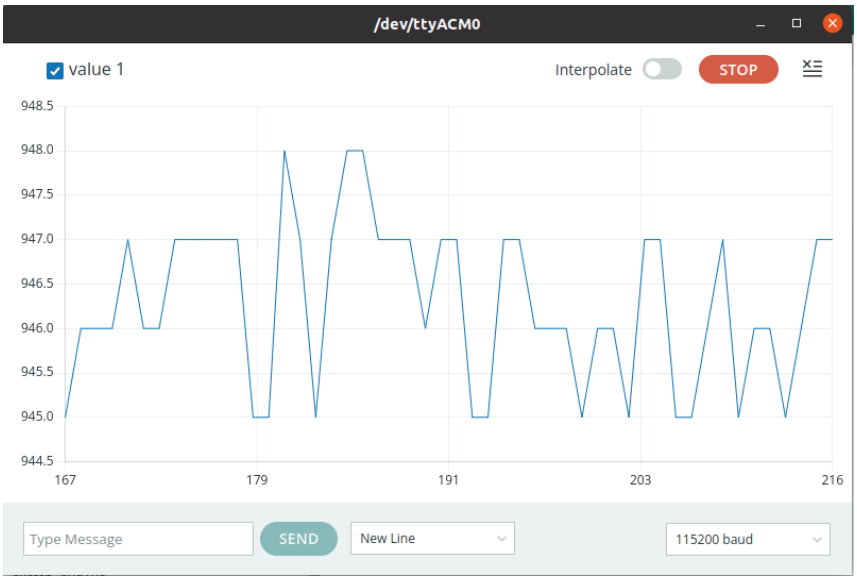
Měření napětí je realizováno funkcí `analogRead()`, fyzicky je k této funkci použita periferie ADC. Zde je možné se setkat s jednou zvláštností, ATmega disponovala 10-bitovým ADC, STM32 disponují obvykle 12-bitovým ADC (méně často 16- a 14-bitovým). Přesto návratová hodnota je i pro STM32 0 ... 1023 (realizováno odběrem 12-bitové hodnoty a bitovým posunem (pro zpětnou kompatibilitu)). Pro nastavení 12-bitového rozlišení je třeba zavolat funkci `analogReadResolution(12)` (dle dokumentace na arduino.cc).

Jednoduchý příklad vypisující na UART hodnoty naměřené na `PA0` může vypadat následovně:

```
1  HardwareSerial mySerial(PA3, PA2);//Rx,Tx
2  void setup() {
3    mySerial.begin(115200);
4    analogReadResolution(12);
5    pinMode(PA0, INPUT);
6  }
7
8  void loop() {
9    //mySerial.print("PA0 value: "); //odokmentovat pro vypis
10   mySerial.println(analogRead(PA0));
11   delay(200);
12 }
```

Listing 5: Analogové čtení

Samotné Arduino nabízí možnost vykreslovat průběh těchto hodnot do jednoduchého grafu pomocí **Tools→Serial Plotter**, viz obrázek 5.



Obrázek 5: Vykreslený graf naměřených hodnot

Jako každé postupně-aproximační ADC je zásadní parametr doba odběru vzorku (nejdelší pro

dosažení vyšší vstupní impedance). Tento parametr je v STM32duinou nastaven jako kompromis mezi rychlostí a impedancí, pro zvýšení je vhodné upravit, viz kapitola 1.4.2.

1.3.4 Analogový zápis, DAC

Na pinech, které disponují DAC převodníkem (je vhodné ověřit v datasheetu), je možné zapisovat hodnotu napětí funkcí `analogWrite()`. Vstupními argumenty funkce `analogWrite()` jsou jméno pinu a hodnota (s rozsahem 0-255). Jednoduchý příklad může vypadat následovně:

```
1 void setup() {
2   pinMode(PA5, OUTPUT);
3   analogWrite(PA5, 127);
4 }
5 void loop() {}
```

Listing 6: analogový „zápis“

I zde se ukazuje podobná zvláštnost jako v případě ADC, DAC na ATmeze byl 8-bitový DAC, na STM32 je však typicky 12-bitový. Analogicky jako u ADC můžeme toto rozlišení upravit funkcí `analogWriteResolution(12)`.

1.3.5 Funkce `analogWrite()`, PWM

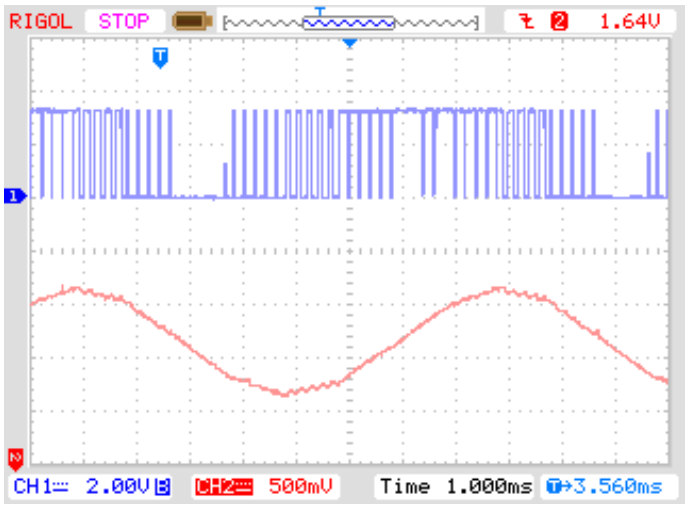
Pokud na pinu není dostupný DAC, ale nějaký čítač je (dle datasheetu, viz kapitola 1.5.1), funkce `analogWrite()` na cílovém pinu použije čítač ke generování obdélníkového signálu se střední hodnotou úměrnou hodnotě vstupního argumentu (PWM po vyfiltrování odpovídá hodnotě, kterou by generoval DAC). Jako příklad je zde uvedeno generování průběhu podobného sinu:

```
1  unsigned char sine[256];
2
3  void setup() {
4    for(int i=0; i<256; ++i) sine[i]=(sin(i*2*PI/255.)+1.0)*255/2;
5    //predpocitani sinu, 256 hodnot
6  }
7
8  void loop() {//perioda modulovaneho signalu 0.256 s
9    unsigned char c=0;
10   analogWrite(PA6, sine[c++]);// PWM out, given duty
11   delay(1);
12 }
```

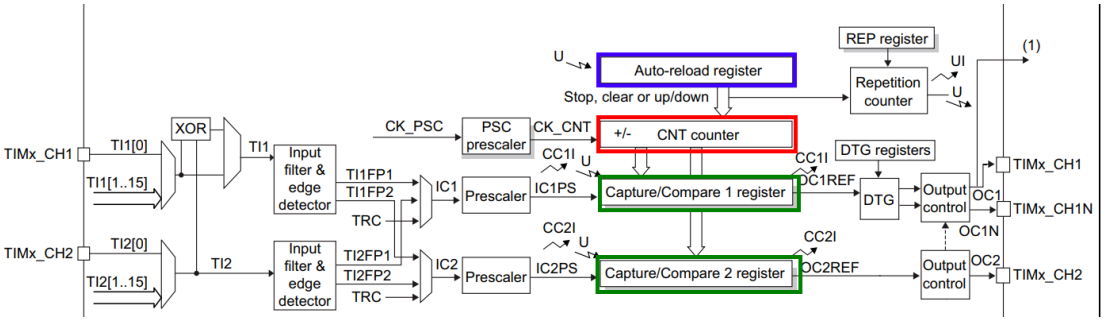
Listing 7: Generování průběhu podobného sinu

Průběh podobný sinu lze zobrazit na osciloskopu, na obrázku 6 je vidět výstup čítače (kanál 1, žlutý) a výstup čítače vyfiltrovaný integračním RC článkem (kanál 2, modrý).

Frekvence výstupu funkce `analogWrite()` je v STM32duinu typicky 1000 Hz. Frekvenci lze změnit funkcí `analogWriteFrequency()`. Při změně frekvence na více pinech je důležité v datasheetu ověřit, které čítače jsou používány (funkce TIMx_CHy kapitola 1.5.1 jak číst datasheet, zároveň je třeba brát v úvahu i negované (označené jako _N) výstupy (kanál a negovaný sdílí hodnotu Capture/Compare registru)). Preferovanou alternativou k funkci `analogWriteFrequency()` může být použití knihovny **HardwareTimer** (viz kapitola 1.3.7).



Obrázek 6: Průběh generovaného obdélníkového signálu, následně upraveného



Obrázek 7: Zjednodušený funkční nákres čítače, převzato z RM0444 (převzato z [4], upraveno)

Podrobnější popis fungování čítače

Z principu jeden čítač s více kanály může generovat na každém kanálu obdélníkové signály různé střídy, ale všechny kanály sdílí jednu hodnotu frekvence (viz obrázek 7). Frekvence je odvozena od hodnoty zapsané v Auto-reload registru (na obrázku 7 zvýrazněno modře), zároveň hodnoty střídy kanálů jsou odvozeny od hodnot v Capture/Compare registrech (na obrázku 7 zvýrazněno zeleně).

Pokud na pinu není DAC, ani čítač, zapíše se hodnota digitálně (prahována oproti polovině číselného rozsahu vstupního argumentu).

1.3.6 Funkce `tone()`

Další funkcí Arduina generující obdélníkový signál je `tone()`. Tato funkce je uváděna pouze pro úplnost, jedná se o generování obdélníkového signálu se střídou 1:1, zadané frekvence. Funkce `tone()` má jedno zásadní omezení (vyplývající z Arduina), je možné generovat v jednu chvíli signál pouze na jednom pinu, což vždy nepostačuje. Zároveň funkce `tone()` není založená na kanálu čítače daného pinu, ale na jiném čítači (určeném v definicích STM32duina, viz kapitola 1.4.2), který realizuje danou funkcionalitu pomocí přerušení, což je méně přesné, typicky při vysokých frekvencích. Přestože je tato funkce často používána v příkladech Arduina, v STM32duinu je velmi doporučeno použít jiné funkce, konkrétně knihovnu **HardwareTimer**.

1.3.7 Knihovna HardwareTimer

Knihovna **HardwareTimer** STM32duina nabízí pokročilejší možnosti nastavení čítačů STM32, než Arduino. Na githubu lze nalézt základní dokumentaci, ve které jsou vypsány funkce třídy a některé příklady. Jednoduchý příklad s generováním PWM s knihovnou může vypadat takto (z datasheetu dle kapitoly 1.5.1 byl vybrán pin PA0 na kterém se nachází TIM2_CH1):

```
1  HardwareTimer htim2 = HardwareTimer(TIM2);
2  void setup() {
3    htim2.setPWM(1, PA0, 100, 50); //kanal 1, pin PA0, 100Hz, 50%
       duty
4  }
5
6  void loop(){}
```

Listing 8: **HardwareTimer**, generování PWM

Čítač je však periferie, kterou lze využít i jinak, např. ke generování periodické události (přerušení). Toho dosáhneme tak, že k čítači „připneme“ funkcí `attachInterrupt()` funkci, tzv. callback (konkrétní formát realizace mechanizmu převzat z Arduina). Příklad, který tlakovým mechanismem bliká LED, lze vidět níže:

```
1  HardwareTimer htim2 = HardwareTimer(TIM2);
2
3  callback_function_t callback(){
4    if(digitalRead(PA5)==LOW) digitalWrite(PA5,HIGH);
5    else digitalWrite(PA5,LOW);
6    return 0;
7  }
8
9  void setup() {
10   pinMode(PA5, OUTPUT);
11   htim2.setOverflow(10, HERTZ_FORMAT);//10 Hz frekvence volani
       callbacku
12   htim2.attachInterrupt(callback);
13   htim2.resume();//spustit citac
14 }
15 void loop(){}
```

Listing 9: **HardwareTimer**, interrupt s callbackem

Protože STM32 disponuje čítači jinými, než původní ATmega, typicky nelze pro práci s čítači použít většinu knihoven pro Arduino (např. oblíbená čítačová knihovna **TimerOne** je napsaná přímo "nad registry"čítače ATmegy).

1.3.8 SPI, I2C, knihovny

SPI a I2C jsou rozšířená rozhraní pro komunikaci mezi mikropočítači. Obě tato rozhraní jsou v Arduinu podporována, stejně tak i v STM32duino. Zmíněná rozhraní jsou velmi často používaná pro připojení rozličných modulů a zařízení, např. externích pamětí, LCD, chytrých senzorů.... Pro většinu takovýchto modulů a zařízení již existují knihovny pro Arduino a obrovským benefitem STM32duina je, že tyto knihovny jsou použitelné i s ním.

Rozhraní SPI

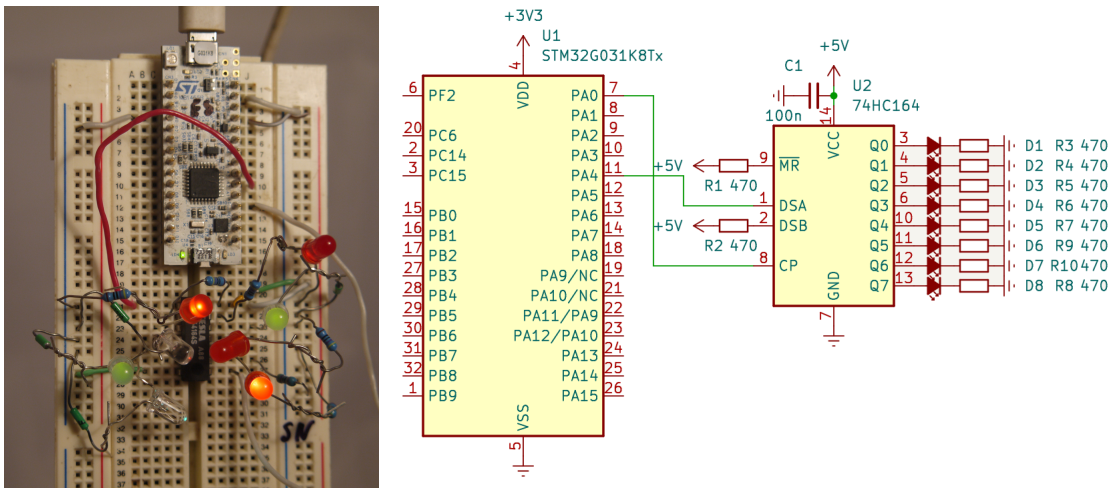
V typické formě používá čtyři signály, MOSI, MISO, SCK, CS. Jedná se o synchronní rozhraní, hodiny generuje uzel master signálem SCK, signálem MOSI posílá data master a ve stejnou chvíli signálem MISO posílá data slave. Signálem CS (volitelný) určuje uzel master, který slave je aktivní.

Použití SPI v Arduinu není v zásadě složité, v první řadě je třeba vložit knihovnu `<SPI.h>`. Konkrétní „výměna bytů“ je realizována funkcí `transfer()`, kdy vstupním argumentem je odeslaný byte, zároveň přijatý byte je vrácen (jako návratová hodnota). Obdobně jako pro UART (v kapitole 1.3.2) je vhodné si pro SPI vytvořit vlastní objekt `mySPI` odpovídající třídy, kdy jsou definovány použité piny (nutné je definovat validní hodnoty MOSI, MISO, SCK pro periferii (viz kapitola 1.5.1, jak číst datasheet). Pro signál CS (když není použit) je možné použít makro `PNUM_NOT_DEFINED`).

Následující příklad ukazuje komunikaci po SPI pro posuvný registr 74x164 (MOSI pin (připojen k pinu DSA; na pinu DSB trvale logická 1), a SCK (připojen k pinu CP), (schéma zapojení příkladu na obrázku 8, vpravo)). Na výstupy posuvného registru (piny Q1-7) můžeme pro demonstraci připojit LED (obrázek 8, vlevo):

```
1  #include <SPI.h>
2  SPIClass mySPI(PA4, PA3, PA0, PNUM_NOT_DEFINED);//MOSI, MISO, SCK,
       CS
3  void setup() {
4    mySPI.begin();//inicializace
5    mySPI.beginTransaction(SPISettings(250000, MSBFIRST, SPI_MODE3));
       ///freq 250k, poradi, CPOL=1 CPHA=1
6  }
7
8  void loop() {
9    mySPI.transfer(0x55,SPI_TRANSMITONLY);//odesilani 0x55,bez prijmu
10   delay(1);
11 }
```

Listing 10: SPI, posuvný registr



Obrázek 8: Demonstrace SPI s posuvným registr, vlevo fotografie výstupů (4 LED svítí, 4 LED zhasnuté), vpravo schéma zapojení

V předchozím příkladě funkcí beginTransaction je nastavena frekvence přenosu (250 kHz, nejpomalejší možná při 64MHz hodinách periferie (před-děličkou 64M/256)), pořadí bitů (první nejvíce významný) a mód (mód 3: CPOL=1, CPHA=1).

Arduino založené na ATmeze častokrát „trpělo“ na nedostatek periferií pro komunikaci s posuvnými registry, proto existuje softwarová emulace, funkce `shiftOut()`, kterou lze použít i v STM32duinu. Následující příklad demonstruje binární čítač (softwarový), jehož hodnota je postupně posílána do posuvného registru:

```
1  void setup() {
2    pinMode(PA0,OUTPUT);
3    pinMode(PA4,OUTPUT); //inicializace
4  }
5  uint8_t cnt = 0;//promenna realizujici "citac"
6  void loop() {
7    shiftOut(PA4, PA0, MSBFIRST, cnt++); //poslat cnt a inkrementovat
8    delay(100);
9  }
```

Listing 11: SPI, posuvný registr, softwarově

Implementace by měla respektovat původní možnosti Arduina, odpovídající dokumentaci lze nalézt na docs.arduino.cc.

I2C

I2C je rozhraní používající dva signály SDA (data), SCK (hodiny). Rozhraní obsluhuje jeden master uzel, který komunikuje s ostatními zařízeními na základě jejich adres. Podmínkou pro správné fungování je tzv. „drátový součin“ na obou signálech, typicky realizovaný pull-up rezistory.

V Arduinu je při použití třeba vložit knihovnu `<Wire.h>` a vytvořit objekt pro rozhraní (`myI2C`, zde specifikujeme použité piny). Objekt je třeba iniciovat funkcí `begin()`, následně je možné nastavit frekvenci přenosu funkcí `setClock()` (funkce je schopna nastavit pouze hodnoty frekvencí dle standardu (omezení Arduina)). Zasílání na rozhraní I2C začíná funkcí `beginTransmission()`, samotné vysílání je realizováno funkcí `write()`, zasílání je zakončeno funkcí `endTransmission()` (funkce vrací 0, přenos proběhl v pořádku (bit ACK 0)). Příjem je realizovaný za pomoci bufferu – nejdříve je funkcí `requestFrom()` „zažádáno“ o zadaný počet bytů, následně funkcí `read()` jsou data vyčtena z bufferu.

První příklad (I2C scan) demonstruje způsob, jak ověřit, že na rozhraní odpovídá nějaké zařízení na nějaké adrese (pokud nastavuje bit ACK do 0):

```
1  #include <Wire.h>
2  TwoWire myI2C(PB7, PB6);  //SDA,SCL
3  HardwareSerial mySerial(PA3, PA2);//Rx,Tx
4
5  void setup() {
6    mySerial.begin(9600);
7    myI2C.begin();
8    myI2C.setClock(100000);
9    for (uint8_t adresa = 1; adresa < 127; ++adresa) {// vsechny
       adresy
10     myI2C.beginTransmission(adresa);
11     if (!myI2C.endTransmission()) {//ACK LOW
12       mySerial.print("Nalezeno: 0x");
13       mySerial.println(adresa, HEX);
14     }
15   }
16 }
17 void loop() {}
```

Listing 12: I2C scan

Druhý příklad ukazuje zápis a čtení, konkrétně je použit MEMS senzor MPU6050. MPU6050 má mít dle datasheetu v registru na adrese 0x75 zapsánu hodnotu 0x68. Příklad ukazuje, jak tuto hodnotu vyčíst a potvrdit (posláním textu na UART).

```
#include <Wire.h>
TwoWire myI2C(PB7, PB6);  //SDA,SCL
HardwareSerial mySerial(PA3, PA2);//Rx,Tx

void setup() {
  mySerial.begin(9600);
  myI2C.begin();
  myI2C.setClock(100000);//nastaveni frekvence
  uint8_t address=0x68;

  myI2C.beginTransmission(address);
  myI2C.write(0x75);//posilani 1 bytu (adresy-registru)
  myI2C.endTransmission();

  myI2C.requestFrom(address,1);//"zadost" o cteni 1 bytu (z adresy drive zaslane)
  int devID = myI2C.read();//cteni 1 bytu z bufferu
  if(devID == 0x68) mySerial.println("devID OK");// vypis na UART pokud vyctena hodnota odpovida datasheetu
}
void loop() {}
```

Listing 13: I2C MPU6050

Jiným příkladem může být použití IO expandéru PCF8574T (zápis hodnoty 0xf, spodní 4 bity 1):

```
#include <Wire.h>
TwoWire myI2C(PB7, PB6);  //SDA,SCL

void setup() {
  myI2C.begin();
  myI2C.setClock(100000);//nastaveni frekvence
  uint8_t address=0x20;//treba overit, lisi se dle nastaveni a vyrobce
  myI2C.beginTransmission(address);
  myI2C.write(0xf);//posilani 1 bytu honoty IO
  myI2C.endTransmission();
}
void loop() {}
```

Listing 14: I2C IO expander PCF8574T

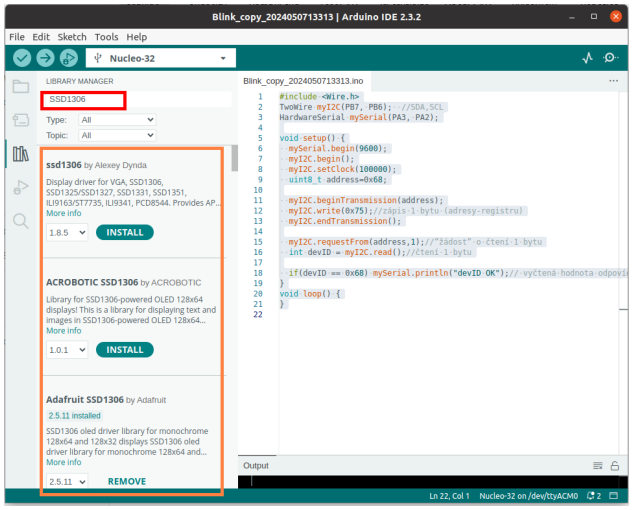
Na githubu lze nalézt krátkou dokumentaci. Původní možnosti zůstávají dle specifikace Arduina, odpovídající dokumentace Arduina se nachází na docs.arduino.cc.

Knihovny SPI, I2C

Dnes však není obvykle potřeba SPI a I2C komunikaci v Arduinu programovat na nízké úrovni, jak ukazovaly předchozí podkapitoly. Pro většinu dostupných zařízení již existují knihovny pro

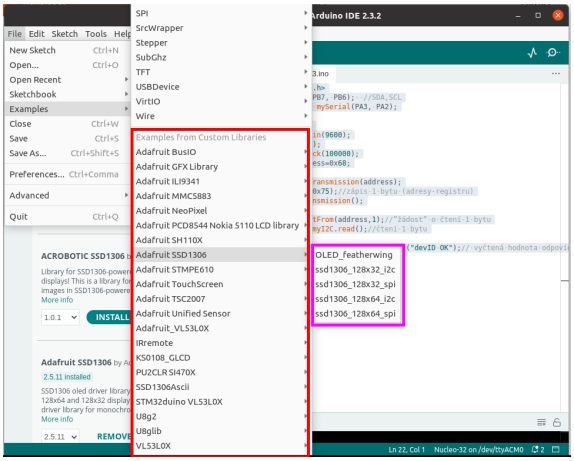
Arduino, které je velmi často možné použít, jak jsou, i s STM32duinem. Tyto knihovny jsou dostupné buď přímo z Arduino IDE, nebo jako .zip archivy (informace o knihovnách je možné nalézt na `https://www.arduino.cc/reference/en/libraries/`).

Instalace knihoven z Arduino IDE je přímočará v nabídce **Tools** → **Manage Libraries...** otevřeme nabídku a pomocí filtru vyhledáme knihovnu a tlačítkem **INSTALL** nainstalujeme (obrázek 9).



Obrázek 9: Výběr knihoven

Pokud pomocí Arduino IDE nenalezneme vhodnou knihovnu, je možné nainstalovat knihovnu i jako archiv pomocí **Sketch** → **Include Library** → **Add .ZIP Library...**. Po instalaci je možné otevřít příklady knihoven v nabídce **File**>**Examples** (obrázek 10), ty mohou být užitečné, když integrujeme nějaký modul do našeho projektu (např. pro prvotní ověření funkčnosti).



Obrázek 10: Výběr příkladu příslušného knihovně

1.3.9 USB

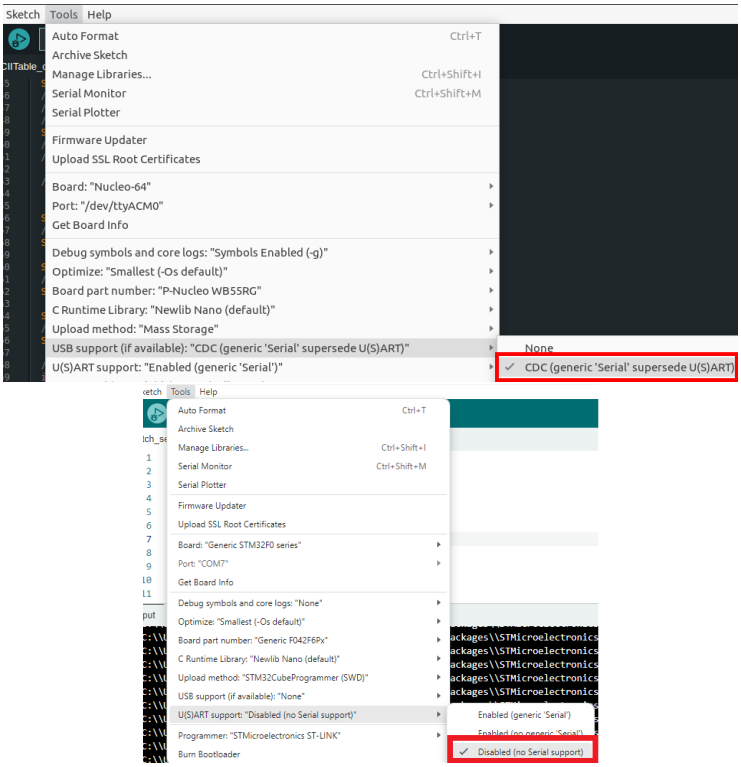
Většina mikrokontrolérů rodiny STM32 disponuje periferií USB (z řady STM32G0 pouze zařízení STM32G0B/C). STM32duino nabízí možnost využít knihovny Arduina pro USB, konkrétně

pro dvě třídy – CDC a HID.

Třída CDC, Virtual COM port

V některých případech může být vhodným řešením použít periferii USB jako COM port namísto periferie USART jako tzv. „Virtual COM Port“.

Funkcionalitu lze nastavit skrze **Tools → USB Support → CDC generic supersede...**, zobrazeno na obrázku 11, vlevo. Po tomto nastavení jsou data výchozí třídy `Serial` předávána přes USB.



Obrázek 11: Nastavení Virtual COM Port

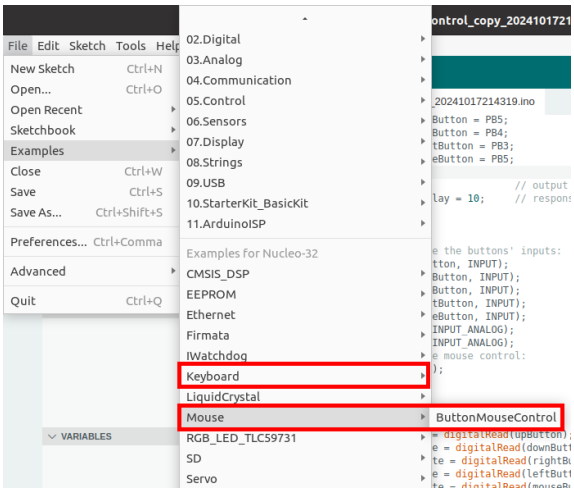
Po nahrání programu by se deska při připojení k USB měla enumerovat jako COM port. Toto řešení má v sobě jistou eleganci (není třeba UART-USB převodník), přesto je zde jistý nedostatek, USB knihovna pro své fungování potřebuje určité množství paměti (okolo 10 kB Flash a 3 kB RAM), které je zejména u mikrokontrolérů s menším množstvím paměti citelné.

Pokud je použit pouze VCP a žádný USART, dává smysl nevkládat knihovny pro tuto periferii (skrze **Tools → U(S)ART support: → Disabled...**) a tím ušetřit na velikosti programu, na obrázku 11, vpravo je ukázáno, jak.

Třída HID, myš, klávesnice

Periferii USB lze pomocí knihoven Arduina použít jako zařízení třídy HID (Human Interface Device), to může být konkrétně klávesnice nebo myš. Chování těchto knihoven má být totožné s Arduinem, při použití této knihovny je dobré vycházet z příkladů **ButtonMouseControl** a **KeyboardMessage** (skrze **File → Examples → Examples for Nucleo-32**, pro rychlou orientaci může pomoci obrázek 12).

Použití příkladu pro myš je přímočaré, pro základní ověření stačí nastavit, které piny jsou použity ve funkci tlačítek. Místo tlačítek lze samozřejmě použít jiné senzory, např. potenciometry



Obrázek 12: Příklady USB

pro osy x a y.

Knihovna pro klávesnici je komplexnější než pro myš, její popis je dobré si před použitím nastudovat `https://docs.arduino.cc/language-reference/en/functions/usb/Keyboard/`. Konkrétně se rozlišuje `write()` (pošle se symbol) a `push()`/`release()`, kdy je klávesa stisknuta/uvolněna.

Pro zadávání textů se knihovna používá vcelku přímočaře (funkci `print()` se předávají texty). Při pokročilejších operacích (např. použití klávesy PrintScreen) lze použít některé ID symbolů dle specifikace (HID Usage Tables [5]). V tom případě je však k ID symbolu třeba přičíst offset 136 a poslat funkcí `write()` (příklad PrintScreen – kód 0x46+136=206). Omezující je však, že vstupní argument je typu `uint8_t`, a tedy některé znaky tak není možné použít (pro ID vyšší než 0x73).

Jako vlastní příklad 15 založený na těchto knihovnách ukazujeme příklad založený na desce 32L476GDISCOVERY, na které se nachází konektor USB a směrový ovladač. V tomto příkladu je směrový ovladač použit jako šipky USB klávesnice (ukázka funkce `Keyboard.write()`).

```
1  #include <Keyboard.h>
2
3  const int up = PA3;
4  const int dn = PA5;
5  const int le = PA1;
6  const int ri = PA2;
7
8  void setup() {
9    pinMode(up, INPUT_PULLDOWN);
10   pinMode(dn, INPUT_PULLDOWN);
11   pinMode(le, INPUT_PULLDOWN);
12   pinMode(ri, INPUT_PULLDOWN);
13   Keyboard.begin();
14 }
15
16 void loop() {
17   if(digitalRead(up)==HIGH) Keyboard.write(KEY_UP_ARROW);
18   if(digitalRead(dn)==HIGH) Keyboard.write(KEY_DOWN_ARROW);
19   if(digitalRead(le)==HIGH) Keyboard.write(KEY_LEFT_ARROW);
20   if(digitalRead(ri)==HIGH) Keyboard.write(KEY_RIGHT_ARROW);
21   delay(100);
```

```
22 }
```

Listing 15: USB HID kurzorové šipky

Příklad 16 lze použít s libovolnou deskou Nucleo-64, která disponuje mikrokontrolérem s USB (uživatel si piny PA11, PA12 musí vyvést na konektor sám). Příklad při stisku tlačítka aktivuje kombinaci kláves ctrl+W (ukázka funkcí `Keyboard.press()` a `Keyboard.releaseAll()`).

```
1  #include <Keyboard.h>
2
3  void setup() {
4    pinMode(PC13, INPUT_PULLDOWN);
5    Keyboard.begin();
6  }
7
8  void loop() {
9    if(digitalRead(PC13)==HIGH){
10     Keyboard.press(KEY_LEFT_CTRL);
11     Keyboard.press('w');
12     Keyboard.releaseAll();
13   }
14   delay(100);
15 }
```

Listing 16: USB HID ctrl+W

Problém PA13 NOE: Na některých mikrokontrolérech STM32 je možné použít typicky pin PA13 ve funkci USB NOE. Tato funkcionalita je specifická a ve většině případů je užitečnější použít pin PA13 jako SWDIO (rozumějte debug). Problém u STM32duina je, že definice (soubor **PeripheralPins.c** – viz popis definicí, kapitola „Definice desek“ 1.4.2) jsou generované z programu STM32CubeMX a při zapnutí USB se nastaví následující piny:

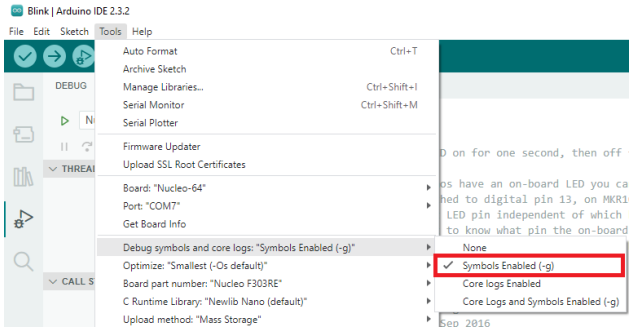
```
WEAK const PinMap PinMap_USB[] = {
  {PA_11, USB, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_NO, GPIO_AF10_USB)}, // USB_DM
  {PA_12, USB, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_NO, GPIO_AF10_USB)}, // USB_DP
  {PA_13, USB, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_UP, GPIO_AF10_USB)}, // USB_NOE
  {PC_9,  USB, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_UP, GPIO_AF10_USB)}, // USB_NOE
  {NC,    NP,  0}
};
```

a jako důsledek nelze použít debug po SWD. To lze však obejít předefinováním WEAK definice pole `PinMap_USB[]` v sketchi typicky takto:

```
const PinMap PinMap_USB[] = {
  {PA_11, USB, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_NO, GPIO_AF10_USB)}, // USB_DM
  {PA_12, USB, STM_PIN_DATA(STM_MODE_AF_PP, LL_GPIO_PULL_NO, GPIO_AF10_USB)}, // USB_DP
  {NC,    NP,  0}
};
```

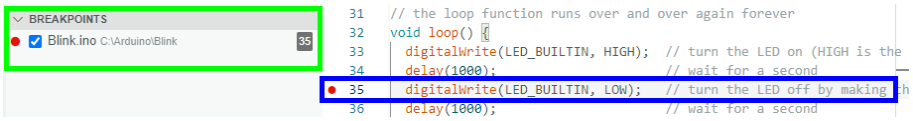
1.3.10 Možnosti debugu ve STM32duinu

V návaznosti na možnosti Arduina, STM32duino nabízí možnosti debugu po SWD využívající ST-Link. To nabízí možnost program zastavovat, krokovat a vyčítat hodnoty registrů. Pro připojení ST-Linkem je třeba nejdříve nahrát zkompilovaný program do desky (např. ikonou **Upload** (→)), zároveň tento program musí obsahovat symboly debugu (nastavení viz obrázek 13). Následné připojení lze provést pomocí ikony **Start Debugging** na řádku ikon.



Obrázek 13: Nastavení symbolů debugu do výstupu kompilátoru

Debug je řešen pomocí OpenOCD (otevřený nástroj používaný v kombinaci se standardním GDB) a nabízí obvyklé možnosti. Obvyklými možnostmi rozumíme breakpointy (program se zastaví v místě umístěného červeného bodu (na obrázku 14 modře zvýrazněno); všechny aktivní breakpointy zobrazeny v seznamu (na obrázku 14 zvýrazněno zeleně)), krokování (pomocí obvyklých ovládacích prvků (na obrázku 15)) a watch výrazy. Navíc je zde využito standardních (dle CMSIS) .svd souborů, díky kterým je možné přehledně zobrazit obsah registrů zařízení (sekce „Cortex peripherals“ v okně, zobrazeno na obrázku 16).



Obrázek 14: Ilustrace použití breakpointu



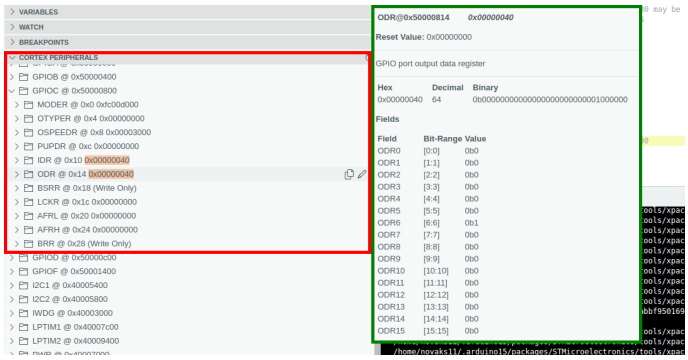
Obrázek 15: Ovládací prvky debugu, zleva **Continue**/**Pause** (spustí/pozastaví běh programu), **Step Over** (vykoná aktuální řádek zastaví na dalším), **Step Into** (zastaví uvnitř funkce volané na řádku), **Step Out** (zastaví při návratu z aktuálně vykonávané funkce), **Restart** (vyvolá reset), **Stop** (ukončí debug)

1.4 Pokročilé možnosti a příklady

1.4.1 Pokročilé možnosti debugu

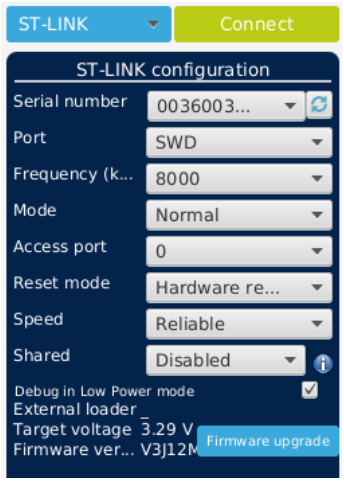
Debug v nástroji STM32Programmer

V návaznosti na debug v STM32duinu se jeví jako vhodné zmínit možnosti debugu v programu STM32CubeProg. Zde probíhá debug bez znalosti struktury programu, kdy jsou vyčítány registry



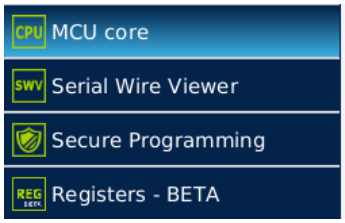
Obrázek 16: Výpis registrů periferií

mikrokontroléru, zároveň jsou zde možnosti, jak do registrů zapisovat, jádro zastavovat a další. V první řadě je dobré se zaměřit na nastavení připojení debuggeru, konkrétně na položku **Mode**.



Mode **Normal** je vhodný pro většinu operací. **Normal** má jeden nedostatek, při připojování typicky vyvolává reset, což v některých případech nemusí být žádoucí; v takovém případě je lepší použít **Mode Hot Plug**, který reset nevyvolává. Posledním modem je **Under reset**, který reset nechává aktivní (to může být výhodné), tento mode však není vhodný, pokud je třeba program z nástroje debuggovat (program se nedostane z obsluhy resetu).

Samotný debug se ovládá ze tří karet (**MCU core**, **Serial Wire Viewer**, **Registers**):



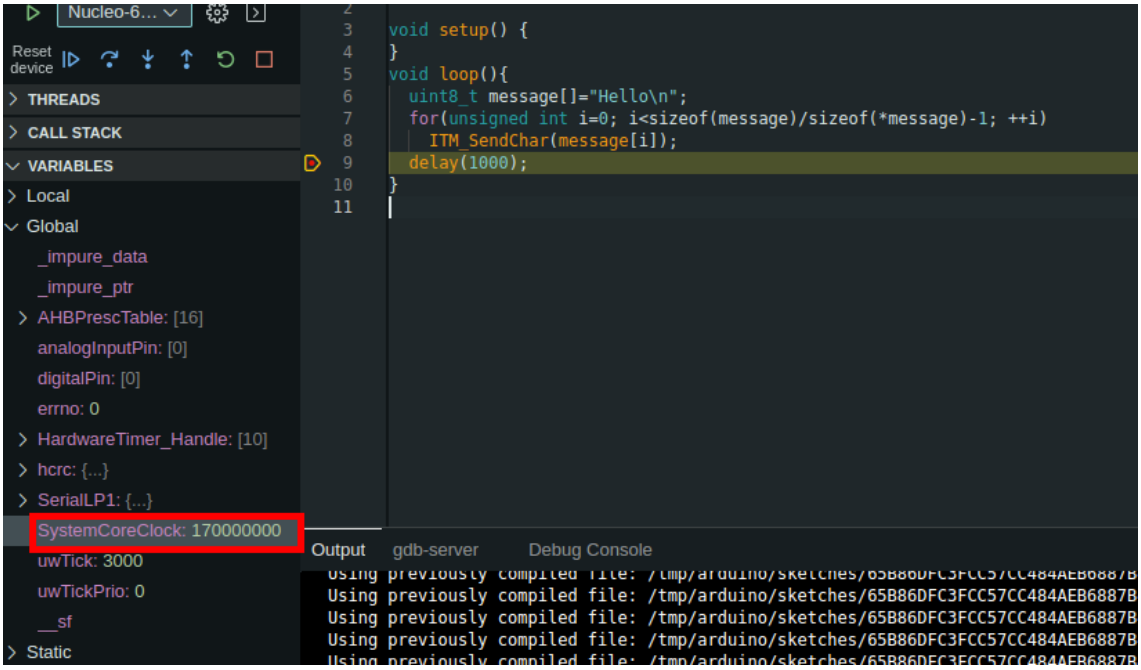
V kartě **MCU core** je možné sledovat a upravovat obsah registrů jádra (případně jádro zastavovat, krokovat, resetovat), v kartě **Serial Wire Viewer** je možné sledovat výpisy po SWO (Cortex-M0/M0+ nepodporuje), v kartě **Registers** lze vyčítat a měnit obsah registrů periferií.

Použití SWO SWO je možností, jak pomocí rozhraní SWD (rozhraní jádra pro debug) realizovat výpisy do konzole (typicky při ladění). Výhodou SWO je nezávislost na periferiích (typicky UART) mikrokontroléru. Použití SWO demonstruje následující příklad napsaný pomocí STM32duina:

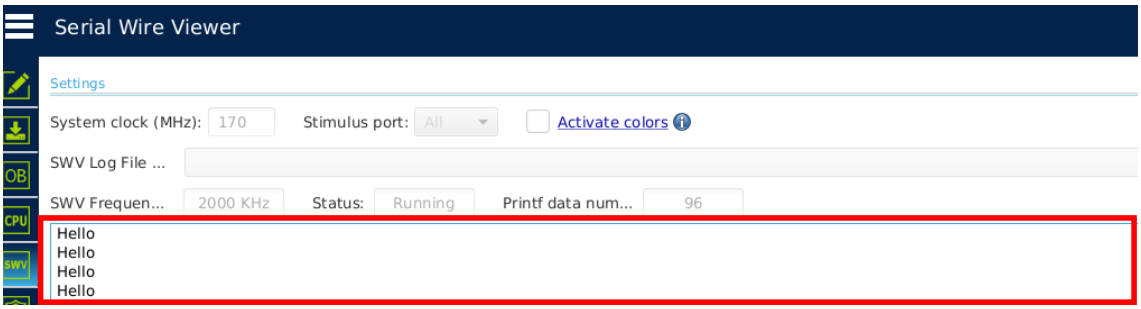
```
1  void setup() {
2  }
3  void loop(){
4    uint8_t message[]="Hello\n";
5    for(unsigned int i=0; i<sizeof(message)/sizeof(*message)-1; ++i)
6      ITM_SendChar(message[i]);
7    delay(1000);
8  }
```

Listing 17: SWO jednoduchý výpis

Program nahrajeme a připojíme se ST-linkem z Arduino IDE. Pro STM32CubeProg budeme potřebovat znát frekvenci hodin, ty vyčteme jako proměnnou `SystemCoreClock`, zde 170 MHz.



Odpojíme (pomocí tlačítka **Stop**) ST-link od Arduino IDE a připojíme se z STM32CubeProg, zadáme frekvenci hodin a spustíme zobrazení dat; v textovém poli bychom měli vidět výpisy:



Podrobnější popis všech funkcionalit STM32CubeProg lze nalézt v UM2237 [6]. Další pokročilé možnosti debugu jsou popsány v aplikační nótě AN4989 [7] (popsáno pro standardní vývojové nástroje).

Možnosti GDB pomocí okna Debug Console

GDB je standardní debugovací nástroj. Při otevření okna **Debug Console** (ikona >) je možné zadávat přímo příkazy. Pomocí příkazů lze dosáhnout dále, než pouze grafickým rozhraním z Arduino IDE, např. je možné krokovat kód po instrukcích příkazem `ni` (ne po řádcích v C). Použití příkazu `ni` je výhodné u příkladů s inline assemblerem, kdy z pohledu C je celý kód v inline assembleru „jedním řádkem“.

ST-link

Pro úplnost, dává smysl se dodatečně zmínit o ST-linku a jeho možnostech. ST-link je proprietární debugger společnosti ST, lze použít výlučně na ladění mikrokontrolérů STM32, případně STM8. ST-Link jako takový je dostupný v několika verzích založených na různých mikrokontrolérech STM32, zde uvádíme některé:

- **ST-Link V1**: Stará verze debuggeru (současné softwarové nástroje již nepodporují) založená na STM32F103C8/STM32F103CB. Lze ji nalézt na nejstarších vývojových deskách STM32 (např. STM32VLDISCOVERY). S verzemi V2 sdílí mikrokontrolér, ale liší se zapojením. Z principu lze drátkováním a přehráním firmware „upgradovat“ na V2 (po internetu kolují návody, od kterých se ST distancuje).
- **ST-Link V2** – aktuální verze debuggeru vhodná pro většinu aplikací:
  - **ST-Link V2.0**: Buď jako samostatný debugger (obsahuje ESD ochranné obvody na vstupech), případně na některých převážně starších vývojových deskách (namísto verze V2.1). Samostatný debugger, typicky podporuje debug STM32 po SWD a JTAG, dále STM8 po SWIM. Zařízení je založeno na mikrokontrolérech STM32F103C8/STM32F103CB.
  - **ST-Link V2.0 ISOL**: Zařízení je založeno na mikrokontrolérech STM32F103C8/STM32F103CB, oproti V2 obsahuje izolaci signálů pomocí optočlenů (typicky vhodné pro některé výkonové aplikace). Dobré je přečíst si před zapojením návod UM1075, konkrétně dbát na zapojení zemního pinu (oproti „neizolované“ variantě). Měl by mít stejné možnosti jako „neizolovaný“ ST-Link V2.0.
  - **ST-Link V2.1**: Zařízení je založeno na mikrokontrolérech STM32F103CB, typicky se nachází na vývojových deskách. Oproti V2.0 nepodporuje JTAG ani SWIM. Oproti V2.0 poskytuje navíc „Virtual COM port“ (funkce USB/UART převoníku, pro uživatelské výpisy) a MCO (8MHz hodinový výstup).
- **ST-Link V3** – oproti V2 poskytující více možností (rychlejší SWD rozhraní, Access porty pro složitější struktury jader):
  - **STLINK-V3SET**: Zařízení poskytující velké množství rozhraní (debug STM32 SWD/JTAG, STM8 SWIM, VCP, CAN, „bridge“...). Debugger je rozšiřitelný o připojitelné („sandwitchovatelné“) desky, např. o desku s optočleny pro izolaci. Debugger je založen na mikrokontroléru STM32F723IE.
  - **STLINK-V3MINI/STLINK-V3MINIE/STLINK-V3MODS**: Debuggery je založen na mikrokontroléru STM32F723IE, jedná se o minimalistické verze pro uživatelské aplikace (pomocí konektoru/jako modul při-pájitelný do uživatelské aplikace).
  - **STLINK-V3PWR**: Zařízení založeno na STM32H7 poskytující mimo debugu možnost měření spotřeb (i malých proudů). Měření spotřeb je analogické zařízení X-NUCLEO-LPM01A s kterým sdílí ovládací software pro měření.

Vývojové desky často nabízejí možnost použití ST-Linku V2.1 pro cílovou aplikaci (zmíněno např. v návodu UM1724[8]). Mimo těchto existují i neoriginální verze, od které se ST distancuje:

- **ST-link V2 „dongle“** Jedná se o minimální zapojení ST-Linku V2 (obrázek) používajících typicky klon mikrokontroléru STM32F103CB. V současnosti (2024) lze tyto upgradovat na aktuální firmware a dokonce používat se softwarovými nástroji od ST (v některých případech mohou nastávat problémy, typicky založené na problémech použitých klonů mikrokontrolérů). Zajímavé, že existuje více verzí DPS s různým rozložením vývodů (často se lze setkat s kabelovými vývody na uživatelských deskách, které díky tomu nemusí být vzájemně kompatibilní).



Další možnosti debugu

Mimo nastíněných možností je možné používat i tradičnější metody k odlaďování programu (uvedeno pro úplnost). První jsou výpisy pomocí periferie UART (viz kapitola 1.3.2), nebo pomocí SWO (kapitola 1.4.1). Obsah výpisu závisí na uživateli, typické je vypisovat hodnoty proměnných, případně registrů mikrokontroléru.

Druhou tradiční metodou může být použití LED a tlačítka, velmi dobře zpracovaná je problematika pro mbed (mechanismus lze aplikovat samozřejmě i jinde) v bakalářské práci L. Bielesch [9], kapitola 6.1. Základní myšlenkou je „zastavování-se“ v programu na určitých místech pro lokalizaci problému (mechanismus velmi podobný breakpointu). Toho lze dosáhnout tak, že na určitých místech v programu jsou umístěna místa kam je vložena část kódu, která rozbliká LED. Takto část kódu (blikání LED) je vykonávána dokud není stisknuto tlačítko, příklad takovéto části kódu je uveden níže.

```
1   while(digitalRead(PC13)==HIGH){//cekani na stisk tlacitka
2     digitalWrite(PA5, HIGH);
3     delay(100);
4     digitalWrite(PA5, LOW);
5     delay(100);
6   }
7   while(digitalRead(PC13)==LOW);//cekani na uvolneni tlacitka
```

Listing 18: Debug LED s tlačítkem

1.4.2 Struktura balíku STM32duino

Adresářová struktura souborů balíku

Pro hlubší pochopení funkcí STM32duina a pro možnosti přizpůsobení programů je vhodné prozkoumat strukturu balíku. Konkrétně se jedná o popis adresářové struktury a rolí jednotlivých souborů. Soubory balíku lze nalézt v adresáři (na Windows):
%loclappdata%/Arduino15/packages/STMicroelectronics/,
obsah adresáře je typicky:

- **hardware/stm32/2.8.1** – základní adresář balíku (obsah odpovídá gitu Arduino_Core_STM32)
  - **CI**, **cmake**, **debugger**, **tools** – složky obsahují skripty zajišťující funkce balíku, např. build, debug ...
  - **cores** – Složka obsahuje implementaci „jazyka Wiring“ pro STM32.
  - **libraries** – Složka obsahuje knihovny portované z Arduina (např. **USBDevice**, **Wire**).
  - **system** – Složka obsahuje drivery (CMSIS a HAL) a middleware STM32 (knihovny pro USB). Dále jsou zde pro každou řadu STM32 nastavení driverů.
  - **variants** – Dle řad STM32 uspořádané definice desek (důležité, popsáno dále).
  - **boards.txt** – Soubor obsahující parametry definovaných desek. Desky obsažené v tomto souboru se zobrazí v UI Arduina.
  - **keywords.txt** – Seznam rezervovaných názvů (pinu, periferii...) pro kontrolu při buildu.
  - **package.json** – Data pro zobrazeni při instalaci z **Manažera desek**.
  - **platform.txt** – Nastavení compileru, buildu a podobně.
- **tools** – Složka obsahující převážně podpůrné soubory.
  - **CMSIS** – CMSIS knihovny
  - **STM32_SVD** – SVD definice pro debug
  - **STM32Tools** – Nástroje převážně pro nahrávání programu
  - **xpack-arm-none-eabi-gcc** – GCC pro kompilaci
  - **xpack-openocd** – OpenOCD pro debug

Pro konkrétní úpravy může být výhodné upravovat především obsah složky **variants**. Ve specifických případech může dávat smysl opravovat obsah složek nastavení ve složce **system**, knihoven **libraries** a implementace „jazyka Wiring“ **cores**. .

Definice desek ve složce variants, soubor boards.txt, a soubory jednotlivých variant

V základním adresáři (**hardware/stm32/2.8.1**) se typicky nachází soubor **boards.txt** obsahující základní definici desek. Desky obsažené v souboru se zobrazí v nabídce v Arduino IDE jako možná cílová deska. Obdobně jako v Arduino IDE jsou definovány skupiny desek (např. „Nucleo_32“, nebo „GenG0“ a dále jednotlivé desky (např. „Nucleo G031K8“). Součástí definice desky je jméno, max. velikosti flash a RAM (`maximum_size`, `maximum_data_size`) pro build, cestu k „**variantě**“ (rozvinuto dále) a odpovídající `.svd` soubor. Dále zde lze nalézt flagy kompilátoru specifické pro danou desku.

V podadresáři `variants` lze nalézt jednotlivé **varianty**. „Varianta“ obsahuje několik souborů, příkladem takového může být `/variants/STM32G4xx/ G431R(6-8-B)(I-T)_G441RB(I-T)` (analogicky pro jinou desku). Ve „variantě“ lze nalézt několik souborů, jejich účel je následující:

- **boards_entry.txt** – Soubor s definicí, ze které lze vycházet při definici vlastní desky v souboru **boards.txt**
- **CMakeLists.txt** – Soubor sloužící při buildu k automatickému vkládání zdrojových kódů (CMake).

- **generic_clock.c** – Soubor typicky obsahuje funkci inicializace hodin. Pokud je třeba upravit lze použít program STM32CubeMX (nastavit hodny zde a odpovídající vygenerovanou část kódu pouze zkopírovat).
- **ldscript.ld** — Linkerscript, standardní GCC (`https://ftp.gnu.org/old-gnu/Manuals/ld-2.9.1/html_chapter/ld_3.html`). Lze použít linkerscript generovaný programem STM32CubeMX s úpravami (je vhodné nastavit velkosti pamětí jako makra (`maximum_size`, `maximum_data_size`) ze souboru **boards.txt**).
- **PeripheralPins.c**: Soubor obsahuje definice (formou polí) s „alternativními funkce“ jednotlivých pinů. V tomto souboru je třeba ověřit, jaké jméno pinu využít při využití určité funkce dle datasheetu (viz dále následující kapitola).
- **PinNamesVar.h** – Soubor obsahuje definice zajišťující alternativní funkce pinů.
- **variant_NUCLEO_G431RB.cpp**, **variant_generic.cpp**: Soubor obsahuje definice obsahující přiřazení původních pinů Arduina (`D0-Dx` a `A0-Ay`) pinům mikrokontroléru (formou pole). Dále se zde může nacházet funkce nastavující hodiny (alternativně k souboru **generic_clock.c**).
- **variant_NUCLEO_G431RB.h**, **variant_generic.h**: Soubor obsahuje definice přiřazení původních pinů Arduina (`D0-Dx` a `A0-Ay`) pinům mikrokontroléru (formou maker). Dále jsou zde uvedeny definice pro výchozí `Serial, I2C, SPI`, timer pro funkci `tone()` a další (definice `BUILTIN_LED` na desce...).

Úpravy definic: pokud třeba upravovat definice v souborech *.c není nutné upravovat definice, ale pouze ve sketchi předefinovat (položky definované zpravidla se symbolem WEAK).

Soubor PeripheralPins.c

Soubor **PeripheralPins.c** obsahuje definice alternativních funkcí pinů. Definice jsou realizovány formou pole, např. pro piny ADC je definováno pole `PinMap_ADC[]`. Z těchto polí lze odvozovat, které periferie jsou pro danou funkcionalitu použity. Pro některé piny může nastat, že je možné danou funkcionalitu realizovat více perifériemi (např. pokud je na mikrokontroléru více ADC, na pinu PA0 lze číst kanálem 1 použít typicky u všech), je pin uveden vícekrát (např. PA_0 a PA_0_ALT1, pro každou takovou možnost bude využitá jiná periferie). V tomto případě je třeba si dát pozor, které piny jsou v kódu využívány (zkontrolovat v souboru).

Mimo alternativních funkcí existují tzv. remapované piny. Remapované piny jsou hardwarovou funkcionalitou některých STM32. Pomocí registrů lze nastavit, který typicky ze dvou pinů (např. `PA9`, nebo `PA11`) bude připojen na fyzický vývod. V definicích STM32duina jsou takto použité piny označeny `_R` , např. `PA_9_R`; balík při kompilaci by měl být schopen tyto piny správně nastavit.

Soubory variant_NUCLEO_G431RB.h, variant_generic.h a problém s názvy pinů

V header souboru definice desky lze najít přiřazení názvu (např. PA0) pinů unikátní číselné hodnotě (int). Tato zdánlivá nelogičnost vychází ze způsobu jak funkce Arduina zacházejí s pinem (každý pin má přiřazené číslo, dle fyzického uspořádání desky D0-13 např). **Jako argument většiny funkcí používejte název pinu právě v tomto formátu (PA0)**.

Aby situace nebyla jednoduchá, tak existuje jiný, racionálnější způsob číslování pinů – PA_0. Tento zápis pevně definuje jako `PA_0 = (PortA << 4) + 0x00`, tedy spodní čtyři bity reprezentují číslo pinu, a „číslo“ portu (od 0<<4 pro GPIOA). Funkce si typicky převádějí automaticky z PA10 → PA_10 dle pole digitalPin[]. **Problém může nastat pokud zaměníme reprezentace PA_0/PA0, prosím berte v úvahu.**

Pro alternativní funkce používejte makra dle header souboru (PA0_ALT1).

build_opt.h

V sekci Wiki na githubu STM32duina se často naráží na soubor `build_opt.h`. Tento soubor v předchozích verzích (v1) Arduina IDE nabízel přijatelnou možnost, jak upravovat vlastnosti knihoven STM32duina (pomocí maker) tato makra byla použita se stejnou syntaxí jako flagy kompilátoru (např. pro zapnutí UARTU `-DHAL_UART_MODULE_ENABLED`).

Problémem tohoto řešení je, že v aktuální verzi se cachují object fily a změny `build_opt.h` se projeví jen při kompilaci knihoven (nekontrolují se a nepropagují se změny). Soubor by bylo možné snadno používat, ale jen při vymazání těchto object filů při každém buildu. Jako možná cesta k „čistému buildu“ je přenastavení optimalizace v nabídce STM32duina (to typicky donutí balík zkompilovat knihovny znovu). Druhou alternativou je vytvoření definice vlastní desky a přidat flagy tam (následující kapitola 1.4.2).

Příkladem použití je přenastavení délky odběru vzorku ADC. Doba odběru je nastavena na mezi STM32duinu 8.5 a 19.5 hodin ADC; závisí na mikrokontroléru. Tato hodnota může být prodloužena nastavením makra `ADC_SAMPLINGTIME` která bude použita jako hodnota odpovídajícího pole bitů v odpovídajícím registru (např. pro STM32G0 pole `SMPx` v registru `ADC_SMPR`). Toho lze dosáhnout pomocí souboru `build_opt.h`, konkrétně přidáním `-DADC_SAMPLINGTIME=7` do něj (pro STM32G0 nastavení nejdelší možné, 160.5 cyklů hodin ADC).

Pro úplnost je vhodné zde zmínit, že pomocí maker (která se dají upravovat pomocí flagu kompilátoru `-D`) jsou konfigurovatelné i samotné HAL drivery (`https://github.com/STM32duino/Arduino_Core_STM32/wiki/HAL-configuration`).

Definice vlastní desky

V případě, že je vhodné definovat vlastní desku, je dobré si přečíst odpovídající stránku na githubu (`https://github.com/STM32duino/Arduino_Core_STM32/wiki/Add-a-new-variant-%28board%29`). Důvodem pro vytvoření definice vlastní definice může být např. problematické nastavení knihoven pomocí maker (jako náhrada použití souboru `build_opt.h`). V tom případě (nastavení knihoven makry) je vhodné si přečíst tuto stránku (github).

Dalším důvodem k vytvoření definice vlastní desky může být úprava velikostí pamětí (flash a RAM jako parametrů `maximum_size`, `maximum_data_size`).

Úprava sekcí paměti (linkerscript)

Pokud je třeba změnit rozložení RAM, nebo Flash (např. požadujeme uživatelské sekce), správným krokem je upravit linkerscript. To může být výhodné, když uživatel chce rezervovat určité místo a nestojí o to, aby to zajistil kompilátor. Linkerscript pro GCC je standardizovaný (dokumentace na webu `https://ftp.gnu.org/old-gnu/Manuals/ld-2.9.1/html_chapter/ld_3.html`). Při úpravě uspořádání paměti je vhodné do pole `MEMORY` definovat vlastní sekci (např. `MY_RAM`), té přiřadit výchozí adresu a délku. Zároveň je třeba náležitě upravit existující sekce. Výsledek může vypadat takto (posledních 16 bytů RAM si uživatel rezervuje):

```
1 /* Memories definition */
2 MEMORY
3 {
4   RAM    (xrw)    : ORIGIN = 0x20000000,   LENGTH =
      LD_MAX_DATA_SIZE-16
5   FLASH    (rx)    : ORIGIN = 0x8000000 + LD_FLASH_OFFSET, LENGTH =
      LD_MAX_SIZE - LD_FLASH_OFFSET
6   MY_RAM    (xrw)    : ORIGIN = 0x20000000+LD_MAX_DATA_SIZE-16,
      LENGTH = 16
7 }
```

Listing 19: Úprava linker filu

1.4.3 Programování s STM32duino s využitím HAL a CMSIS definic

V tuto chvíli dává smysl se podívat na komponenty, na kterých je STM32duino založeno, motivací může být použití těchto komponent v Arduino IDE k rozšíření možností. Konkrétně se jedná o knihovny HAL od ST a Arm GNU toolchain. S knihovnami HAL je možné vytvořit efektivnější program, který bude do velké míry reflektovat hardwarové uspořádání STM32. Součástí HAL jsou i CMSIS definice, pomocí nichž je možné pracovat přímo nad registry mikrokontroléru. Další možností je využít možností GCC a vkládat do programu napsaném v C části napsané v assembleru.

Použití funkcí STM32 HAL

ST, jako výrobce mikrokontroléru, dodává knihovny HAL, které by měly pokrývat vše, co periferie STM32 nabízí. Jelikož je STM32duino na těchto knihovnách založeno, je možné jejich funkce volat. Knihovny HAL jsou dostupné pro každou řadu STM32 jako součást STM32Cube (např. STM32CubeG0) balíčků dostupných od výrobce (z st.com), součástí je nápověda ve formátu .chm. Je dobré zmínit, že se nejedná o podporovanou funkcionalitu, ale spíše o využití známých skutečností. Jako příklad je zde uvedeno blikání LED:

```
1  void setup() {
2    __HAL_RCC_GPIOC_CLK_ENABLE();//zapnuti hodin GPIOA
3    GPIO_InitTypeDef GPIO_InitStruct ={GPIO_PIN_6,GPIO_MODE_OUTPUT_PP
       ,
4              GPIO_NOPULL,GPIO_SPEED_FREQ_VERY_HIGH,0};//nastaveni
     pinu
5    HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);//samontna inicializace
6  }
7
8  void loop() {
9    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_6);//zmena logicke urovne PC6
10   delay(100);
11 }
```

Listing 20: Blikání LED HAL

Znalost knihoven HAL může být velkou výhodou při přechodu na pokročilejší IDE, které knihovny HAL může využívat; jedná se např. o STM32CubeIDE (zdarma od výrobce ST), případně komerční IAR EWARM, nebo Keil µVision v kombinaci s STM32CubeMX.

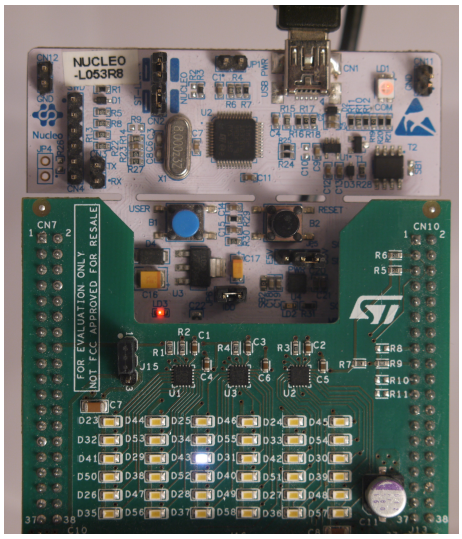
Portovaní příkladu z STM32Cube do STM32duina

STMicroelectronics dodává určité množství rozšiřujících desek pro své vývojové desky Nucleo. K těmto deskám typicky dodává i příklady, které mají definovanou strukturu. Velmi často, než začít vývoj vlastní aplikace „od nuly“, dává smysl vyzkoušet některé dostupné příklady a na jejich knihovnách postavit svou aplikaci.

Jako příklad takového vývoje uvádíme desku STEVAL-LLL007V1 (obrázek 17). Tato deska obsahuje pole LED ovládané pomocí budiče LED1202. K tomuto budiči se nepodařilo autorovi dohledat knihovnu pro Arduino. Přímo k desce STEVAL-LLL007V1 je dodávána jen binárka, ale existuje deska X-NUCLEO-LED12A1 se stejnými řadiči a velice podobným zapojením. K této desce je dodáván balík s příklady X-CUBE-LED12A1. Při bližším prozkoumání příkladů je vidět, že příklad běží na jak je, i na desce STEVAL-LLL007V1.

Samotné portování příkladu se sestává z výběru zdrojových kódů, driverů (knihoven), které příklad používá (překopírovat soubory do adresáře Arduinovského sketche). Dále je třeba vzít soubor `main.c` a převést ho do podoby Arduina (souboru .ino) – obsah funkce `main()` z většiny překopírovat do funkce `setup()` Arduina a obsah cyklu `while(1)` použít ve funkci `loop()`.

Ostatní funkce definované v souboru main.c je třeba překopírovat taktéž do souboru .ino, globální proměnné překopírovat do souboru main.h. Do souboru .ino je třeba překopírovat také funkce



Obrázek 17: Deska STEVAL-LLL007V1 připojená k NUCLEO-L053R8. Na zařízení běží portované demo zkompilované pomocí STM32duina

obsažené v souboru stm32l0xx_hal_msp.c (inicializace pinů použitých periferií). Poté je vhodné zkusit sketch sestavit a vyřešit případné problémy (chybějící definice). K úspěšnému buildu není třeba nahrazovat „HALovské“ funkce „Arduinovskými“, převzaté funkce by měly bez problémů běžet nad knihovnami HAL obsaženými v balíku STM32duino.

Použití funkcí STM32 LL

LL knihovny jsou součástí HAL, jako méně abstraktní alternativa k hlavní části HAL knihoven. Použití v STM32duino je velmi omezené, tyto knihovny nejsou typicky zahrnuty při sestavování programu; jednou z výjimek může být funkce `LL_GPIO_TogglePin()` kterou lze použít (přesto `HAL_GPIO_TogglePin()` je také dostupná).

Úvod do přechodu k nástrojům STM32Cube

STMicroelectronics dodává ke svým mikrokontrolérům software, již zmiňované knihovny HAL. Knihovny HAL dále ST rozšiřuje do balíků STM32Cube podle řad, např. STM32CubeG0. K těmto balíkům je možné stáhnout vývojové prostředí STM32CubeIDE, které obsahuje generátor projektů STM32CubeMX. STM32CubeIDE je prostředí cílené pro profesionální uživatele, přesto s určitou snahou ho může ovládnout i amatér. Veškerý zmíněný software je poskytován zdarma po registraci na stránkách `st.com`.

ST poskytuje jisté množství video-kurzů ohledně ekosystému zabývající se STM32; rozcestník lze nalézt na **st.com**. Pro začátky s STM32CubeIDE je dobré začít kurzem **STM32CubeIDE basics**.

Možnost přístupu k registrům periferií mikrokontrolérů podle CMSIS:

CMSIS je standard definovaný firmou ARM (tvůrcem jader Cortex-M, která mikrokontroléry STM32 používají) obsahující knihovny. Jeho účelem bylo standardizovat knihovny pro mikrokontroléry s jádry ARM Cortex-M pro snadnější portování mezi mikrokontroléry různých výrobců. Jeho součásti jsou definice registrů, knihovny pro DSP, RTOS a další. Zmíněné definice registrů jsou použity i v STM32duinu a můžeme je využít pro nízkoúrovňové programování, příkladem je následující blikání LED:

```
1 void setup() {
```

```
2    RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;//zapnuti hodin GPIOA
3    GPIOA->MODER &= ~GPIO_MODER_MODE5;
4    GPIOA->MODER |= GPIO_MODER_MODE5_0;//nastaveni pinu
5  }
6
7  void loop() {
8    GPIOA->ODR |= GPIO_ODR_OD5;  // zapis log 1
9    delay(100);
10   GPIOA->ODR &= ~GPIO_ODR_OD5; // zapis log 0
11   delay(100);
12 }
```

Listing 21: Blikání LED CMSIS drivery

Pro úplnost a další pochopení je dobré si uvést, jak by vypadal program, ekvivalentní řádek po řádku, bez použití definic:

```
1  void setup() {
2    *(uint32_t*)(0X40021000+0x4C) |= (0b1<<0);//zapnuti hodin GPIOA
3    *(uint32_t*)(0x48000000+0x00) &= ~(0b11<<10);
4    *(uint32_t*)(0x48000000+0x00) |= (0b1<<10);//nastaveni pinu
5  }
6
7  void loop() {
8    *(uint32_t*)(0x48000000+0x14) |= (0b1<<5);  // zapis log 1
9    delay(100);
10   *(uint32_t*)(0x48000000+0x14) &= ~(0b1<<5); // zapis log 0
11   delay(100);
12 }
```

Listing 22: Blikání LED C bez knihoven

Možnost použití kódu napsaného v assembleru:

Vkládání kódu napsaného v assembleru je již krajní možností, v tu chvíli opravdu je program psán „instrukci po instrukci“. Díky tomu můžeme mít opravdu naprostou kontrolu nad jádrem (možná efektivita), však na úkor abstrakce. STM32duino používá ke kompilaci GCC, které nabízí možnost vkládání assembleru do kódu napsaného v C, jedná se o tzv. inline assembler, příkladem může být následující blikání LED:

```
1  //for G03 Nucleo LD3; tailorable to any other initialized GPIO PIN
2  void write_p(uint32_t state, uint32_t gpioAdd, uint32_t pin){
3   // when LSB 1 of "state" write gpio_pin HIGH, else LOW
4   // state output state
5   // gpioAdd as GPIOx base address
6   // pin number of pin e.g. 6 for PA6
7
8   __asm__ volatile (
9     "LDR R4, [R1,#0x14]\n"//load ODR to R4
10    "LDR R3, =1\n" //set R3 1
11    "LSL R3, R3, R2\n"//1<<pin; R2 may be overwritten
12    "BIC R4, R4, R3\n" // clear ODx
13
14    "LDR R2, =1\n"//set R2 1
```

```
15     "TST R0, R2\n"//compare 1 with state value; R0 may be
       overwritten
16     "BEQ WRITE\n"//if state 1 skip setting of OD6
17     "ORR R4, R4, R3\n"//set ODx 1
18
19     "WRITE: STR R4,[R1,#0x14]\n"
20   );
21 }
22
23 void setup() {
24   pinMode(PC6, OUTPUT);
25 }
26 uint8_t a=0;
27 void loop() {
28   write_p(++a, 0x50000800, 6);//used GPIOC as 0x50000800
29   a &=1;
30   delay(200);
31 }
```

Listing 23: Iniline assembly, blikání LED

Předchozí příklad je napsaný pro NUCLEO-G031K8 – jeho problém je přenositelnost, konkrétně adresa brány GPIO nejsou na všech mikrokontrolérech STM32 na stejných adresách. Pro dosažení přenositelnosti dává smysl při kombinaci C a assembleru použít CMSIS drivery a předávat adresu struktury, která odpovídá základní adrese periferie (následující příklad).

```
1  //for G03 Nucleo LD3; tailorable to any other initialized GPIO PIN
2  void write_p(uint32_t state, GPIO_TypeDef* gpio, uint32_t pin){
3   // when LSB 1 of "state" write gpio_pin HIGH, else LOW
4   // state output state
5   // gpio type GPIO_TypeDef* as GPIOx (to COMPLY CMSIS def)
6   // pin number of pin e.g. 6 for PA6
7
8   __asm__ volatile (
9     "LDR R4, [R1,#0x14]\n"//load ODR to R4
10    "LDR R3, =1\n" //set R3 1
11    "LSL R3, R3, R2\n"//1<<pin; R2 may be overwritten
12    "BIC R4, R4, R3\n" // clear ODx
13
14    "LDR R2, =1\n"//set R2 1
15    "TST R0, R2\n"//compare 1 with state value; R0 may be
      overwritten
16    "BEQ WRITE\n"//if state 1 skip setting of OD6
17    "ORR R4, R4, R3\n"//set ODx 1
18
19    "WRITE: STR R4,[R1,#0x14]\n"
20  );
21 }
22
23 void setup() {
24   pinMode(PC6, OUTPUT);
25 }
26 uint8_t a=0;
```

```
27 void loop() {
28  write_p(++a, GPIOC, 6);//used GPIOC as x50000800
29  a &=1;
30  delay(200);
31 }
```

Listing 24: Iniline assembly, blikání LED s využitím CMSIS definic

Dokumentaci k inline assembly v GCC lze nalézt na stránce gcc.gnu.org/onlinedocs, případně v knize od J. Yiu [10] a programming manuálech k používanému jádru, např. PM0223[11]. Předávání parametrů pro jádra Cortex-M je definováno v dokumentu AAPCS (Procedure Call Standard for the ARM Architecture) [12].

Následující příklad ukazuje, jak lze použít inline assembly pro práci s polem a jak předat návratovou hodnotu zpět do C. Zároveň je uvedena intrinsic funkce NOP (může být vhodné k ní přiřadit break point pro demonstraci).

```
1  uint32_t sumArray(uint32_t * array, uint32_t len){
2  // array uint32_t array to be summed
3  // len size of array
4  // return sum of array content
5
6   __asm__ volatile (
7     "MOVS R2,#0\n"
8     "START: LDR R3, [R0]\n"//load ODR to R4
9     "ADD R0,#4\n"//increment pointer
10    "ADD R2,R2,R3\n"
11    "SUB R1,R1,#1 \n"
12    "BNE START\n"//if state 1 skip setting of OD6
13    "MOV R0,R2\n"//use R2 as return value
14  );
15  return (uint32_t) array;//return first argument (R0) that is
      actually "array" pointer
16 }
17
18 void setup() {
19 uint32_t pole[3]={1,2,3};
20   pole[0] = sumArray(pole,sizeof(pole)/sizeof(*pole));
21   __NOP();
22 }
23
24 void loop() {
25 }
```

Listing 25: Iniline assembly, cyklus sčítání hodot v poli

Následující příklad ilustruje rychlé načítání hodnot registru (adresy) do pole:

```
1  void sampleArray(uint32_t * array, uint32_t len, uint32_t* address)
      {
2  // array uint32_t array to be filled
3  // len size of array (elements, not bytes)
4  // address to be sampled to array
5   __asm__ volatile (
6     "START: LDR R3, [R2]\n"
7     "STMIA R0!, {R3}\n"
```

```
8       "SUB R1,R1,#1 \n"
9       "BNE START\n"
10    );
11  }
12
13  void setup() {
14  uint32_t pole[30]={1,2,3};
15    sampleArray(pole,sizeof(pole)/sizeof(*pole),(uint32_t*)&(GPIOA->
        IDR));
16    __NOP();
17  }
18
19  void loop() {}
```

Listing 26: Iniline assembly, cyklus čtení hodnot do pole

Při debugování programů napsaných v inline assembleru se při pohledu z C celá inline sekce jeví jako jeden řádek; pro krokování je výhodné použít příkaz `ni` GDB (viz kapitola „Možnosti GDB pomocí okna **Debug Console**“).

1.5 Specifické vlastnosti balíku, pokyny pro kurzy (LPE, KPE)

V této kapitole jsou popsány některé specifické záležitosti související s nasazením STM32duina ve výuce, které však přímo nesouvisejí s balíkem jako takovým (kapitola Jak číst datasheet, Boot a Option byty). Dále jsou zde uvedeny konkrétní příklady pro předměty a kurzy, kde již byl balík STM32duino využit (Předmět LPE, Kurz KPE, Kurz ETC)

1.5.1 Jak číst datasheet

Datasheet (korektně česky katalogový list) je dokument obsahující klíčové parametry součástky (v tomto případě mikrokontroléru STM32). Mimo elektrických parametrů lze zde nalézt popisy funkcí jednotlivých pinů mikrokontroléru. Znalost těchto funkcí je klíčová pro psaní téměř jakéhokoli programu; pokud např. na pinu není funkce kanálu ADC, není možné tam měřit napětí. Jednotlivé funkce daných pinů jsou uvedeny v tabulce, typicky v kapitole nazvané **Pinouts, pin description and alternate functions**.

Na obrázku 18 je vidět část takovéto tabulky; funkce pinu jsou uvedeny ve sloupci **Alternate functions** a **Additional Functions**. Rozdíl mezi **Alternate** a **Additional** funkcemi je, že může být zvolena vždy jedna **Alternate** funkce (nebo GPIO) a navíc je možné používat **Additional** funkce.

V STM32duinu nastává jedna nekompatibilita – jazyk Wiring neřeší případ, kdy na stejném pinu je možné použít stejnou funkcionalitu různými prostředky (např. pokud je třeba generovat PWM a na pinu jsou dva kanály čítače). Tato skutečnost je vyřešena definováním pinu vícekrát (např. PA_0 a PA_0_ALT1), viz kapitola 1.4.2. Zároveň může nastat opačný problém; funkcionalita je dosažitelná více piny (např. stejný kanál čítače na dvou různých pinech), přesto mikrokontrolér toto nepodporuje – je proto třeba ověřovat v datasheetu a souboru **PeripheralPins.c**, které piny jsou kdy použity k dosažení dané funkcionality.

1.5.2 Bootloader

Bootloader je program obsažený ve specifické paměti mikrokontroléru (systémové flash); jeho účelem je nahrávání uživatelského programu. Pro vstup do bootloaderu je třeba splnit určitou „boot konfiguraci“, která určuje, jak se bude daný mikrokontrolér chovat po připojení napájení (z jaké paměti má bootovat). Pro jednoduchost berme nejstarší mikrokontrolér rodiny STM32,

Pin						Pin name (function upon reset)	Pin type	I/O structure	Note	Alternate functions	Additional functions
SO8N	WLCSP18	TSSOP20	UFQFPN28	LQFP32 / UFQFPN32	LQFP48 / UFQFPN48						
-	D4	11	10	11	-	PA4	I/O	FT_a	-	SPI1_NSS/I2S1_WS, SPI2_MOSI, TIM14_CH1, LPTIM2_OUT, EVENTOUT	ADC_IN4, TAMP_IN1, RTC_TS, RTC_OUT1, WKUP2
-	C3	12	11	12	16	PA5	I/O	FT_ea	-	SPI1_SCK/I2S1_CK, TIM2_CH1_ETR, LPTIM2_ETR, EVENTOUT	ADC_IN5
-	E3	13	12	13	17	PA6	I/O	FT_ea	-	SPI1_MISO/I2S1_MCK, TIM3_CH1, TIM1_BK, TIM16_CH1, LPUART1_CTS	ADC_IN6

Obrázek 18: Tabulka s popisy pinu

STM32F1. Tento měl definované chování dvěma piny BOOT0 a BOOT1. Pro oba piny v logické 0 mikrokontrolér nabootuje z paměti flash. Při BOOT0=1 a BOOT1=0 mikrokontrolér nabootuje ze systémové flash (bootloaderu). Pro oba piny v logické 1 mikrokontrolér nabootuje z paměti RAM.

Obdobné chování je implementováno prakticky na všech mikrokontrolérech řady STM32, jen pro uspoření pinů jsou často dva piny nahrazeny jedním, případně ten jeden lze použít jako GPIO (a „odnastavit“ pinu funkci BOOT0, typicky pomocí option bytů). Konkrétní nastavení mikrokontroléru pro vstup do bootloaderu lze dohledat v AN2606[2]. Pro mikrokontroléry STM32G0 a STM32C0 je pin BOOT1 nahrazen option bitem **nBoot1** a pin BOOT0 je buď určen option bitem **nBoot0**, nebo pinem **PA14-BOOT0** (toto určuje další option bit **nBOOT0_SEL**). Pro mikrokontroléry STM32G4 je pin BOOT1 nahrazen option bitem **nBoot1** a a pin BOOT0 je buď určen option bitem **nBoot0**, nebo pinem **PB8-BOOT0** (toto určuje další option bit **nSWBoot0**).

1.5.3 Option byty

Option byty jsou uložené ve speciální části paměti flash a určují jistá pokročilá nastavení mikrokontroléru (například ochrany proti vyčtení paměti debuggerem). Pro každé zařízení se liší sestava option bytů, jejich hodnoty z výroby jsou k nalezení v referenčních manuálech mikrokontrolérů. Operace **Mass erase** která smaže celou uživatelskou paměť flash, hodnoty option bytů nemění.

Pro tuto práci (pro mikrokontroléry STM32G0 a STM32G4) jsou zajímavé option byty ze skupiny „User configuration“, konkrétně option byty související s konfigurací bootu (viz předchozí kapitola 1.5.2). Druhým zajímavým option bytem je **NRST_MODE** na mikrokontrolérech STM32G0 a STM32G4 určuje, jestli NRST pin má funkci resetu (nebo má funkci odpovídajícího GPIO).

1.5.4 Toggle pin test

Pro srovnání rychlosti jednotlivých částí funkcí STM32duina byl proveden tzv. „Toggle pin test“ – měření změny hodnot digitálního výstupu pro různé implementace. Příkladem takového programu může být ve variantě pro Arduino program 27.

```
1 void setup() {
2   pinMode(PA5, OUTPUT);
3 }
4
```

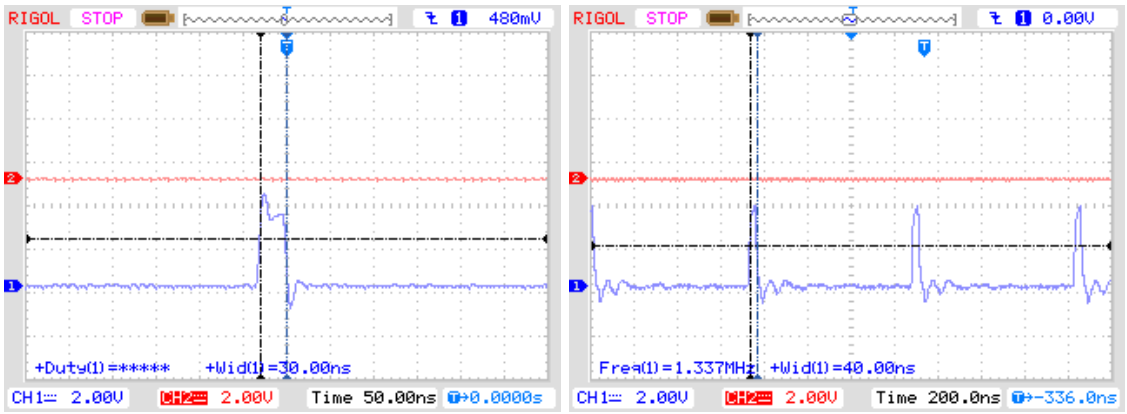
```
5  void loop() {
6    digitalWrite(PA5, HIGH);
7    digitalWrite(PA5, LOW);
8  }
```

Listing 27: Toggle test arduino

Výsledky jsou následující (při 64MHz systémových hodinách) jsou uvedeny v tabulce 1. Známou zajímavostí je, že překlad CMSIS driverů je o jednu instrukci pomalejší než zápis přímo do paměťového prostoru. Však při takto rychlém toggle testu je citelnější než samotný zápis do paměti návrat a opětovné volání funkce `loop()` (viditelné na průbězích osciloskopu 19).

implementace	čas HIGH	čas LOW	odpovídající kód (při zakomentování delay)
Arduino	2.5 μs	2.1 μs	27
ST HAL	0.6 μs	1.3 μs	20
CMSIS struct	3 cykly	0.7μs	21
přímo C	2 cykly	0.7μs	22

Tabulka 1: Výsledky toggle test



Obrázek 19: Ukázka naměřeného průběhu pro C zápisem na adresu

1.5.5 Poznámky k použití STM32duina v B3B38LPE

Příklad použití magnetometru MMC5883

V předmětu B3B38LPE je využíván tříosý magnetometr MMC5883. V současnosti existuje knihovna od firmy Adafruit [13] dostupná jako .zip archiv na githubu. Knihovnu je možné přidat pomocí "Sketch"→"Include Library"→"Add .ZIP Library..."(ke knihovně je nutné nainstalovat závislosti (další knihovny); ty jsou již dostupné z Arduino IDE). Pro začátek je dobré si otevřít příklad dodávaný s knihovnou (v příkladech "Adafruit MMC5883"→"magsensor") a rozšířit ho o mapování pinů SDA a SCL. Základní příklad (alespoň dle chování) nepoužívá "flipování", proto je zde uvedeno následující příklad (modifikovaný příklad "magsensor") (výstup tří os je možné zobrazit pomocí "Tools"→ "Serial Plotter").[1]

```
1  #define INTERNAL_CONTROL_0    0x08
2  #define FLIP_SET 0x08
3  #define FLIP_RESET 0x10
```

[1]Před přepsáním upraveného příkladu **upozornění:** PB8 může být na STM32G431 použit jako **BOOT0 pin**.

```
4
5  #include <Adafruit_MMC5883.h>
6
7  Adafruit_MMC5883 mag = Adafruit_MMC5883(12345);
8
9  void setup(void)
10 {
11   Serial.begin(115200);
12   Wire.setSDA(PB_9);
13   Wire.setSCL(PB_8);//Warning: PB8 may be also BOOT0 pin on G4
14   if(!mag.begin())
15   {
16     Serial.println("ERROR: magnetometer not found");
17     while(1);//break
18   }
19   mag.setContinuousFreq(MMC5883_CMFREQ_5HZ);//set frequency
20 }
21 void loop(void)
22 {
23   /* Get a new sensor event */
24   sensors_event_t event1,event2;
25   mag.getEvent(&event1);
26
27   Wire.beginTransmission(0x30);//set flipping pulse
28   uint8_t cmd[2] = {INTERNAL_CONTROL_0,FLIP_SET};
29   Wire.write(cmd,2);
30   Wire.endTransmission();
31   delay(400);
32
33   mag.getEvent(&event2);
34   Wire.beginTransmission(0x30);//reset flipping pulse
35   cmd[1]=FLIP_RESET;
36   Wire.write(cmd,2);
37   Wire.endTransmission();
38
39   /* Display the results can be used with arduino Serial plotter*/
40   Serial.print((event1.magnetic.x-event2.magnetic.x)/2);
41   Serial.print(" ");
42   Serial.print((event1.magnetic.y-event2.magnetic.y)/2);
43   Serial.print(" ");
44   Serial.println((event1.magnetic.z-event2.magnetic.z)/2);
45   /* "Compass" by atan2 function in degrees for xy
46   Serial.println(180*atan2(event1.magnetic.y-event2.magnetic.y,
       event1.magnetic.x-event2.magnetic.x)/PI);*/
47   delay(400);
48 }
```

Listing 28: Příklad použití magnetometru

Příklad s PWM ovládané po UARTu

V předmětu B3B38LPE byl dále prezentován vzorový příklad s PWM napsaný v mbedu. Jeho port pro STM32duino může vypadat následovně:

```
#define SINE_STEPS 8        // Number of dutycycle steps for output wave
#define SINE_OUT_FREQ 2000  // Frequency of output sine in Hz
//#define PI 3.141592f        // Constants to compute the sine waveform
#define SINE_STEPS_RAD (2.0f * PI / (float)SINE_STEPS)
float sine_duty[SINE_STEPS];                        // Table to generate the sine waveform using dutycycles
#define PWM_FREQ 16000                              // Frequency of Pulse Width Modulated signal in Hz
HardwareSerial Serial2(PA3_ALT1, PA2_ALT1);                  // UART2 TX RX
HardwareTimer pwmGen = HardwareTimer(TIM1);  //PA9
HardwareTimer pwm_ticker = HardwareTimer(TIM3);


// Ticker calls this fucntion to update the PWM dutycycle
callback_function_t pwm_duty_updater() {
  static int idx = 0;
  pwmGen.pause();
  pwmGen.setPWM(2, PA9, PWM_FREQ, sine_duty[idx] * 100);  //channel ,pin Set the dutycycle % to next value in array
  idx++;
              // Increment the idx
  if (idx == SINE_STEPS)
    idx = 0;  // Reset the idx when teh end has been reached
  return NULL;
}
unsigned int aread0;
unsigned int aread1;
char buffer[20];
char comIN;
unsigned char status = 0;
float v1;
float v2;
float tim = 0;
unsigned char i;
void setup() {
  pinMode(PA8, OUTPUT);
  pinMode(PA0, INPUT_ANALOG);
  pinMode(PA1, INPUT_ANALOG);
  analogReadResolution(16);

  //pwm_ticker.setMode(1, TIMER_OUTPUT_COMPARE, NC);//by now not use
  pwmGen.setMode(2, TIMER_OUTPUT_COMPARE_PWM1, PA9);
  Serial2.begin(9600);  //9600 baud rate
  for (i = 0; i < SINE_STEPS; i++) {
    sine_duty[i] =
      ((sin(i * SINE_STEPS_RAD)) * 0.8f + 1.0f) / 2.0f;  // convert sine (-1.0 .. +1.0) into dutycycle (0.0 .. 1.0)
```

```
43    }
44    Serial2.printf("PWM test & measure!\r\n");
45  }
46
47  void loop() {
48    if (Serial2.available()) {  // je k dispozici novy prikaz?
49      comIN = Serial2.read();
50      switch (comIN) {  // a podle toho co prislo, proved danou akci
51        case '0':       // not sending data
52          status = 0;
53          Serial2.printf("Off\r\n");
54          break;
55        case '1':  // sending data
56          status = 1;
57          Serial2.printf("On\r\n");
58          break;
59        case 'a':  // LED off
60          digitalWrite(PA8, LOW);
61          Serial2.printf("LED Off\r\n");
62          break;
63        case 's':  // LED on
64          digitalWrite(PA8, HIGH);
65          Serial2.printf("LED On\r\n");
66          break;
67        case 'q':  // generate PWM 2kHz
68          Serial2.printf("2 kHz PWM On\r\n");
69          pwm_ticker.pause();
70          pwmGen.pause();
71          pwmGen.setPWM(2, PA9, 200, 50);
72          break;
73        case 'r':  // generate PWM 2kHz
74          Serial2.printf("16 kHz PWM On\r\n");
75          pwm_ticker.pause();
76          pwmGen.pause();
77          pwmGen.setPWM(2, PA9, 16000, 50);
78          break;
79        case 'w':  // stop generating PWM
80          pwm_ticker.pause();
81          pwmGen.pause();
82          pwmGen.setPWM(2, PA9, 16000, 0);
83          Serial2.printf("Off\r\n");
84          break;
85        case 'e':
86          pwm_ticker.pause();
87          pwmGen.pause();
88          pwmGen.setPWM(2, PA9, 12500, 50);
89          delay(1);
90          pwmGen.setPWM(2, PA9, 12500, 0);
91          Serial2.printf("Burst\r\n");
92          break;
93        case 'n':
94          pwmGen.pause();
```

```
95          pwmGen.setPWM(2, PA9, (float)PWM_FREQ, 50);
96          pwm_ticker.pause();
97          pwm_ticker.attachInterrupt(pwm_duty_updater);
98          pwm_ticker.setPWM(1, NC, (SINE_STEPS * SINE_OUT_FREQ)/2000,
        50);//freq divided by 2000 for real-time oscillosope check
99          Serial2.printf("2kHz PWM sine On\r\n");
100         break;
101       case 'm':
102         pwm_ticker.pause();
103         pwmGen.setPWM(2, PA9, 12500, 0);
104         Serial2.printf("Off\r\n");
105         break;
106       default:  // kdyz prijde neznamy znak, provede se "default"
      cast kodu
107         break;
108     };  // switch
109   };    // if
110   aread0 = analogRead(PA_0);
111   aread1 = analogRead(PA_1);
112   ;
113   aread0 /= 20;  // hruby prevod na mV...(65535/3300)
114   aread1 /= 20;
115   v1 = aread0 / 1000.0f;
116   v2 = aread1 / 1000.0f;
117   if (status) {
118     tim += 0.05;
119     Serial2.printf("%04u,%04u\r\n",aread0,aread1); //SerialChart
120     //Serial2.printf("$$P%.2f,%.3f,%.3f;", tim, v1, v2);  // Data
      Plotter by Jiri
121     // Maier
122   }
123   delay(50);
124 }
```

Listing 29: Portovaný příklad z mbedu

SSD1306 po SPI

SSD1306 je v současnosti dobře dostupný displej vhodný pro levné aplikace. V předmětu B3B38LPE je použit pro demonstraci použití SPI. Jako demonstrace nízkoúrovňové komunikace po SPI je uveden následující příklad, nejdříve je poslána iniciační sekvence nastavující displej, dále je posílán samotný obsah vykreslený na displej. Obsah je generován s využitím iterátoru cyklu, kdy je použit ternární operátor (generuje pruhy na displej dvou šířek).

```
1  #include <SPI.h>
2
3  uint8_t initSTR[] = {0xAE, 0xD5, 0x80, 0xA8, 63, 0xD3, 0x0, 0x40, 0
       x8D, 0x14, 0x20, 0x00, 0xA1, 0xC8, 0xDA, 0x24, 0x81, 0xCF, 0xD9,
        0xF1, 0xDB, 0x40, 0xA4, 0xA6, 0xAF);
4
5  #define RST PC7
6  #define CS PA9
7  #define DC PB6
```

```
8  #define DIN PA7
9  #define CLK PA5
10
11 void command(uint8_t c) {
12   digitalWrite(CS, HIGH);
13   digitalWrite(DC, LOW);
14   digitalWrite(CS, LOW);
15   SPI.transfer(c);
16   digitalWrite(CS, HIGH);
17 };
18
19 void data(uint8_t c) {
20   digitalWrite(CS, HIGH);
21   digitalWrite(DC, HIGH);
22   digitalWrite(CS, LOW);
23   SPI.transfer(c);
24   digitalWrite(CS, HIGH);
25 };
26
27 void setup() {
28   Serial.begin(9600);
29   pinMode(RST, OUTPUT);
30   pinMode(CS, OUTPUT);
31   pinMode(DC, OUTPUT);
32
33   digitalWrite(RST, LOW);
34   delay(1);  //ms delay for being sure
35   digitalWrite(RST, HIGH);
36
37   SPI.setMOSI(DIN);
38   SPI.setSCLK(CLK);
39   SPI.begin();
40
41  //initialization commands, same sequence as by i2c
42  for(int i=0;i<sizeof(initSTR)/sizeof(*initSTR);++i) command(
      initSTR[i]);
43 }
44
45 void loop() {
46   command(0x0 | 0x0);  // low col = 0
47   command(0x10 | 0x0);  // hi col = 0
48   command(0x40 | 0x0); // line #0
49   digitalWrite(CS, HIGH);
50   digitalWrite(DC, HIGH);
51   digitalWrite(CS, LOW);
52   for(int i=0; i<1024; i++) SPI.transfer(i&4 ? 0xff :0x00);
53   digitalWrite(CS, HIGH);
54
55   delay(2000);
56
57   command(0x0 | 0x0);  // low col = 0
58   command(0x10 | 0x0);  // hi col = 0
```

```
59    command(0x40 | 0x0); // line #0
60    digitalWrite(CS, HIGH);
61    digitalWrite(DC, HIGH);
62    digitalWrite(CS, LOW);
63    for(int i=0; i<1024; i++) SPI.transfer(i&2 ? 0xff :0x00);
64    digitalWrite(CS, HIGH);
65
66    delay(2000);
67  }
```

Listing 30: SSD1306 po SPI

PCD8544 po SPI

Tento příklad používá další oblíbený řadič displejů PCD8544. Tento řadič používá jiný protokol, než SSD1306. Obsah příkladu je však obdobný předchozímu (generování „pruhů“ rozdílné šířky).

```
1   #include <SPI.h>
2
3   #define RST PC7
4   #define CE  PA9
5   #define DC  PB6
6   #define DIN  PA7
7   #define CLK  PA5
8
9   void LcdWriteData(uint8_t dat)
10  {
11    digitalWrite(DC, HIGH); //DC pin is high for data
12    digitalWrite(CE, LOW);  //CS set low
13    SPI.transfer(dat); //transfer the data
14    digitalWrite(CE, HIGH); // get CS back high
15  }
16
17  void LcdWriteCmd(uint8_t cmd)
18  {
19    digitalWrite(DC, LOW); //DC pin is low for commands
20    digitalWrite(CE, LOW);
21    SPI.transfer(cmd);
22    digitalWrite(CE, HIGH);
23  }
24
25  void setup()
26  {
27    pinMode(RST, OUTPUT);
28    pinMode(CE, OUTPUT);
29    pinMode(DC, OUTPUT);
30    digitalWrite(RST, LOW);
31    delay(1);/
32    digitalWrite(RST, HIGH);
33
34    SPI.setMOSI(DIN);
35    SPI.setSCLK(CLK);
36    SPI.begin();
```

```
37
38    LcdWriteCmd(0x21);  // LCD extended commands
39    LcdWriteCmd(0x80|0x20);  // set LCD Vop (contrast)
40    LcdWriteCmd(0x04);  // set temp coefficent
41    LcdWriteCmd(0x14);  // LCD bias mode 1:40
42    LcdWriteCmd(0x20);  // LCD basic commands
43    LcdWriteCmd(0x0C);  // LCD normal video
44  }
45  void loop()
46  {
47
48    for(int i=0; i<504; i++) LcdWriteData(i&4 ? 0xff :0x00); // reset
        LCD
49    delay(2000);
50    for(int i=0; i<504; i++) LcdWriteData(i&2 ? 0xff :0x00); // set
       LCD
51    delay(2000);
52  }
```

Listing 31: PCD8544 po SPI

SSD1306 po I2C

Displej s řadičem SSD1306 mimo SPI může být připojen i pomocí I2C. Následující program je ekvivalentní příkladu v kapitole „SSD1306 po SPI“, jen je použité jiné komunikační rozhraní (I2C namísto SPI).

```
1  #include <Wire.h>
2  uint8_t initSTR[] = {//string for initialization of lcd
3   0xAE,0x20,0x00,0xB0,0xC8,0x00,0x10,0x40,0x81,0xFF,0xA1,0xA6,0xA8,0
      x3F,
4   0xA4,0xD3,0x00,0xD5,0xF0,0xD9,0x22,0xDA,0x12,0xDB,0x20,0x8D,0x14,0
      xAF
5  };
6
7  void setup() {
8    Wire.begin();
9    Wire.beginTransmission(0x3C);//zacatek prenosu
10   Wire.write(initSTR, sizeof(initSTR));//"bufferovani" zpravy
11   Wire.endTransmission();//konec prenosu, nasleduje odeslani
12 }
13
14 void loop() {
15   delay(1000);
16   for (uint8_t i = 0; i < 64 / 8; i++) {
17     Wire.beginTransmission(0x3C);
18     Wire.write(0x40);//write data
19     for (uint8_t j = 0; j < 128; j++) Wire.write(j & 4 ? 0x0 : 0xff
       );
20     Wire.endTransmission();
21   }
22
23   delay(1000);
```

```
24    for (uint8_t i = 0; i < 64 / 8; i++) {
25      Wire.beginTransmission(0x3C);
26      Wire.write(0x40);//write data
27      for (uint8_t j = 0; j < 128; j++) Wire.write(j & 8 ? 0x0 : 0xff
        );
28      Wire.endTransmission();
29    }
30  }
```

Listing 32: SSD1306 po I2C

Pro ovládání SSD1306 na vyšší úrovni existuje řada knihoven často dostupných z Arduino IDE, za zmínku stojí SSD1306Ascii – minimalistická a disponující příklady pro velké množství konfigurací (rozměrů displeje). Tato konkrétní knihovna podporuje jak I2C, tak SPI. Arduinovské knihovny pro zařízení připojená po I2C nebo SPI obvykle bez problémů na STM32duinu (používají se typicky knihovní funkce Arduina).

1.5.6 Materiály ke kurzu KPE

V kurzu praktické elektroniky (pro studenty nastupující do prvního ročníku v běhu 2024) bylo používáno STM32duino jako náhrada prostředí mbed používaného v předchozích letech. Příklady vycházely z obsahu tohoto dokumentu (LED, tlačítko, PWM, sinus pomocí PWM). Výukové materiály (prezentační snímky) jsou součástí příloh práce.

1.5.7 Materiály pro kurz ETC

Pro účely kurzu ETC byly vytvořeny prezentace k použití prostředí Ardublock v kombinaci s prostředím STM32duino. Prvotních běhů (2023) bylo používáno Arduino IDE verze 1.8.6 v kombinaci s balíkem Ardublock ve verzi, která lze stáhnout volně z webu. Tato verze je omezena na pojmenování pinů původního Arduina (DX a AX), tento nedostatek byl vyřešen převodní tabulkou. Pro následné běhy byl Ardublock upraven pro pojmenování libovolným textovým literálem (aby bylo možné používat názvy pinů STM32, např. PA0).

Do Arduino IDE verze 2 není možné přidat Ardublock jako rozšíření, proto bylo vhodné upravit Ardublock do samostatně spustitelné formy, která bude generovat z grafického rozhraní kód (ve formátu .ino). Program je spustitelný z příkazové řádky (nebo souboru .bat) jako:

```
java -jar ardublock-all.jar
```

současnou verzi lze nalézt na githubu (zdrojové kódy a spustitelný soubor).

2 Literatura

Reference

[1] F. Pillon, "Getting started." `https://github.com/STM32duino/Arduino_Core_STM32/wiki/Getting-Started`, 2024.

[2] STMicroelectronics, "An2606 – application note, stm32 microcontroller system memory boot mode," 2024. Rev 64, dostupné na webu.

[3] Arduino, "arduino-examples." `https://github.com/arduino/arduino-examples/tree/main`, 2024.

[4] STMicroelectronics, "Rm0444 – reference manual, stm32g0x1 advanced arm®-based 32-bit mcus," 2024. Rev 6, dostupné na webu.

[5] USB-Implementers'-Forum, "Hid usage tables for universal serial bus (usb)," 2022. Version 1.5.

[6] STMicroelectronics, "Um2237 – user manual, stm32cubeprogrammer software description," 2024. Rev 26, dostupné na webu.

[7] STMicroelectronics, "An4989 – application note, stm32 microcontroller debug toolbox," 2021. Rev 3, dostupné na webu.

[8] STMicroelectronics, "Um1724 – user manual, stm32 nucleo-64 boards (mb1136)," 2020. Rev 14, dostupné na webu.

[9] L. Bielesch, "Optimální využití mbed ide v laboratorní výuce," 2019. BP ČVUT , FEL.

[10] J. Yiu, *The Definitive Guide to ARM® Cortex®-M0 and Cortex-M0+ Processors*. Amsterdam ; Boston: Newnes, 2015.

[11] STMicroelectronics, "Pm0223 – programming manual, stm32 cortex®-m0+ mcus programming manual," 2024. Rev 9, dostupné na webu.

[12] ARM, "Procedure call standard for the arm® architecture," 2024. dostupné na webu.

[13] Adafruit-Industries, "Adafruit_mmc5883." `https://github.com/adafruit/Adafruit_MMC5883`, 2022.